

UNIVERZITA PARDUBICE

FAKULTA ELEKTROTECHNIKY A  
INFORMATIKY

BAKALÁŘSKÁ PRÁCE

2025

Egor Razboinikov

Univerzita Pardubice  
Fakulta Elektrotechniky a Informatiky

Digitalizace evidence pracovní doby  
Bakalářská práce

# ZADÁNÍ BAKALÁŘSKÉ PRÁCE

(projektu, uměleckého díla, uměleckého výkonu)

|                     |                                     |
|---------------------|-------------------------------------|
| Jméno a příjmení:   | Egor Razboinikov                    |
| Osobní číslo:       | I21219                              |
| Studijní program:   | B0688A140009 Informační technologie |
| Téma práce:         | Digitalizace evidence pracovní doby |
| Zadávající katedra: | Katedra informačních technologií    |

## Zásady pro vypracování

Dokument evidence pracovní doby vyučujícího vyžaduje provázanost rozvrhu s úvazkem vyučujícího při respektování času přestávek. Cílem bakalářské práce je navrhnout, vytvořit, odzkoušet a předat funkční multiplatformní aplikaci jejíž vstupy jsou: rozvrh vyučujícího, čas na oběd, plán dovolených, státní svátky, školní volna a další. Požadovaným výstupem je PDF dokument (tabulka) se všemi náležitostmi potřebnými pro odevzdání. Grafické rozhraní poskytne pohodlné zadávání, umožní dodatečné ruční modifikace a zobrazování náhledu před vygenerováním PDF dokumentu.

Rozsah pracovní zprávy: **min. 30 stran**  
Rozsah grafických prací:  
Forma zpracování bakalářské práce: **tištěná**

Seznam doporučené literatury:

PŠENČÍKOVÁ, J. Algoritmizace. Kralice na Hané, Computer Media, 2009.  
MAREŠ, Martin a VALLA, Tomáš. Průvodce labyrintem algoritmů 2. vydání. Praha: CZ.NIC, 2022. ISBN 978-80-88168-63-8  
PECINOVSKÝ, Rudolf. OOP: naučte se myslet a programovat objektově. Brno: Computer Press, 2010. ISBN 978-80-251-2126-9.

Vedoucí bakalářské práce: **Ing. Radek Matoušek**  
Katedra informačních technologií

Datum zadání bakalářské práce: **15. prosince 2024**  
Termín odevzdání bakalářské práce: **16. května 2025**

L.S.

**prof. Ing. Petr Doležel, Ph.D.** v.r.  
děkan

**Ing. Jan Panuš, Ph.D.** v.r.  
vedoucí katedry

V Pardubicích dne 28. února 2025

Prohlašuji:

Práci s názvem Digitalizace evidence pracovní doby jsem vypracoval samostatně. Veškeré literární prameny a informace, které jsem v práci využil, jsou uvedeny v seznamu použité literatury.

Byl jsem seznámen s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon), ve znění pozdějších předpisů, zejména se skutečností, že Univerzita Pardubice má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je Univerzita Pardubice oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladů, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

Beru na vědomí, že v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů, a směrnicí Univerzity Pardubice č. 7/2019 Pravidla pro odevzdávání, zveřejňování a formální úpravu závěrečných prací, ve znění pozdějších dodatků, bude práce zveřejněna prostřednictvím Digitální knihovny Univerzity Pardubice.

V Pardubicích dne 9. 5. 2025

Egor Razboinikov v.r.

## **PODĚKOVÁNÍ**

Tady chtěl bych poděkovat svému vedoucímu Ing. Radek Matoušek za hezké vedení bakalářské práce a rychle odpovědi. Také rád bych poděkoval svým rodičům a kamarádům za jejich podporu po celou dobu studia.

## **ANOTACE**

Cílem této bakalářské práce je navrhnout, implementovat a ověřit multiplatformní desktopovou aplikaci pro digitalizaci evidence pracovní doby vyučujících. Aplikace automaticky zpracuje data z rozvrhu (STAG), zohlední polední přestávky, plánované nepřítomnosti, státní svátky a pravidla EPD a z nich vytvoří přehledný výkaz. Výstupem je PDF v podobě odpovídající požadavkům fakulty, přičemž vstupní údaje lze pohodlně upravit v intuitivním grafickém rozhraní. Práce popisuje použitou metodiku, volbu technologií a architektury a shrnuje výsledky praktického ověření a testování.

## **KLÍČOVÁ SLOVA**

C#, .NET, Avalonia UI, MVVM, AXAML, ReactiveUI, SQLite, Entity Framework Core, QuestPDF, desktopová aplikace, rozvrh, evidence pracovní doby, generování PDF, automatizace.

## **TITLE**

Digitization of Teachers' Working Time Records

## **ANNOTATION**

The aim of this bachelor thesis is to design, implement, and validate a cross-platform desktop application for digitizing university teachers' working time records. The application automatically processes timetable data (STAG), considers lunch breaks, absences, public holidays, and EPD rules, and produces a clear monthly report. The output is a PDF document compliant with faculty requirements, while the underlying data can be conveniently edited through an intuitive GUI. The thesis presents the methodology, technology and architecture choices, and summarizes the results of practical evaluation and testing.

## **KEYWORDS**

C#, .NET, Avalonia UI, MVVM, AXAML, ReactiveUI, SQLite, Entity Framework Core, QuestPDF, desktop application, timetable, working time records, PDF generation, automation.

# OBSAH

|                                                                                                                                                                                                                                                                                                                                                                                               |    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| SEZNAM ILUSTRACÍ .....                                                                                                                                                                                                                                                                                                                                                                        | 11 |
| SEZNAM ZKRATEK A ZNAČEK .....                                                                                                                                                                                                                                                                                                                                                                 | 12 |
| TERMINOLOGIE .....                                                                                                                                                                                                                                                                                                                                                                            | 14 |
| ÚVOD .....                                                                                                                                                                                                                                                                                                                                                                                    | 15 |
| 1 TEORETICKÁ ČÁST .....                                                                                                                                                                                                                                                                                                                                                                       | 17 |
| 1.1 Evidence pracovní doby.....                                                                                                                                                                                                                                                                                                                                                               | 17 |
| 1.2 Architektura a vzory .....                                                                                                                                                                                                                                                                                                                                                                | 18 |
| 1.2.1 Co je architektura softwaru.....                                                                                                                                                                                                                                                                                                                                                        | 19 |
| 1.2.2 MVVM – Model–View–ViewModel .....                                                                                                                                                                                                                                                                                                                                                       | 19 |
| 1.2.3 MVC – Model–View–Controller.....                                                                                                                                                                                                                                                                                                                                                        | 20 |
| 1.2.4 Monolit, modulární monolit a mikroslužby .....                                                                                                                                                                                                                                                                                                                                          | 21 |
| 1.3 Programovací jazyky .....                                                                                                                                                                                                                                                                                                                                                                 | 21 |
| 1.3.1 Jazyk C#.....                                                                                                                                                                                                                                                                                                                                                                           | 22 |
| 1.3.2 Jazyk SQL.....                                                                                                                                                                                                                                                                                                                                                                          | 22 |
| 1.4 Framework Avalonia UI .....                                                                                                                                                                                                                                                                                                                                                               | 23 |
| 1.5 Vývojové prostředí .....                                                                                                                                                                                                                                                                                                                                                                  | 24 |
| Tato kapitola popisuje nástroje, na kterých projekt stojí, a proč byly zvoleny. Cílem je, aby šel zdrojový kód snadno otevřít, spustit a rozvíjet i na jiném počítači a s odstupem času. Pro vývoj v C#/.NET používám integrované prostředí od Microsoftu a pro lokální databázi samostatný nástroj nad SQLite. Díky tomu je celý řetězec od kódu po data jednoduchý a opakovatelný.[2] ..... | 24 |
| 1.5.1 Microsoft Visual Studio.....                                                                                                                                                                                                                                                                                                                                                            | 24 |
| 1.5.2 SQLiteStudio .....                                                                                                                                                                                                                                                                                                                                                                      | 24 |
| 1.6 Knihovny .....                                                                                                                                                                                                                                                                                                                                                                            | 25 |
| 2 PRAKTICKÁ ČÁST .....                                                                                                                                                                                                                                                                                                                                                                        | 27 |
| 2.1 Programovací jazyky .....                                                                                                                                                                                                                                                                                                                                                                 | 27 |
| 2.1.1 Jazyk C#.....                                                                                                                                                                                                                                                                                                                                                                           | 27 |

|                                                   |    |
|---------------------------------------------------|----|
| 2.1.2 Jazyk XAML.....                             | 28 |
| 2.2 Struktura aplikace .....                      | 29 |
| 2.4 Souborová struktura aplikace.....             | 30 |
| 2.4.1 Složka Behaviors .....                      | 31 |
| 2.4.2 Složka Controls.....                        | 31 |
| 2.4.3 Složka Converters .....                     | 31 |
| 2.4.4 Složka Data .....                           | 31 |
| 2.4.5 Složka Helpers .....                        | 32 |
| 2.4.6 Složka Messages .....                       | 32 |
| 2.4.7 Složka Models.....                          | 33 |
| 2.4.8 Složka Services .....                       | 33 |
| 2.4.9 Složka ViewModels .....                     | 33 |
| 2.4.10 Složka Views .....                         | 33 |
| 2.4.11 Složka Kořenové soubory projektu.....      | 33 |
| 2.5 Struktura databáze.....                       | 33 |
| 2.5.1 Přehled tabulek .....                       | 34 |
| 2.5.2 Shrnutí rozhodnutí .....                    | 36 |
| 2.6 Uživatelské rozhraní aplikace .....           | 36 |
| 2.6.1 Hlavní stránka aplikace.....                | 36 |
| 2.6.2 Dialogové okno pro vytvoření události ..... | 38 |
| 2.6.3 Dialogové okno pro změnu události .....     | 39 |
| 2.6.4 Vizuální prezentace pro den .....           | 40 |
| 2.6.5 Vizuální prezentace pro týden .....         | 41 |
| 2.6.6 Vizuální prezentace pro měsíc .....         | 42 |
| 2.6.7 Globální nastavení .....                    | 43 |
| 2.6.8 Ukázka PDF .....                            | 44 |
| 2.7 Algoritmy.....                                | 45 |

|                                                        |    |
|--------------------------------------------------------|----|
| 2.7.1 Algoritmus vyrovnávání hodin .....               | 45 |
| 2.7.2 Algoritmus importu rozvrhu ze STAG .....         | 49 |
| 2.8 Testování aplikace .....                           | 52 |
| 2.8.1 Cíl a kritéria přijetí.....                      | 52 |
| 2.8.2 Testovací prostředí.....                         | 52 |
| 2.8.3 Metodika .....                                   | 52 |
| 2.8.4 Očekávané rozdíly mezi Windows a Linuxem .....   | 52 |
| 2.8.5 Scénáře (podrobný popis provedeního testu) ..... | 53 |
| 2.8.6 Souhrnná tabulka testů.....                      | 54 |
| 2.8.7 Závěr a známá omezení .....                      | 55 |
| ZÁVĚR .....                                            | 56 |
| POUŽITÁ LITERATURA .....                               | 57 |

# SEZNAM ILUSTRACÍ

|                                                        |    |
|--------------------------------------------------------|----|
| Obrázek 1 Tabulka EPD .....                            | 18 |
| Obrázek 2 Architektura MVVM.....                       | 20 |
| Obrázek 3 Architektura MVC.....                        | 20 |
| Obrázek 4 Souborová struktura aplikace .....           | 30 |
| Obrázek 5 Ukázka kódu třídy AppDbContext.....          | 32 |
| Obrázek 7 Struktura tabulky Events .....               | 34 |
| Obrázek 8 Struktura tabulky GlobalSettings .....       | 35 |
| Obrázek 9 Struktura tabulky UserSettings.....          | 36 |
| Obrázek 10 Levý panel aplikace.....                    | 37 |
| Obrázek 11 Pravý panel aplikace.....                   | 38 |
| Obrázek 12 Dialogové okno pro vytvoření.....           | 39 |
| Obrázek 13 Dialogové okno pro změnu .....              | 40 |
| Obrázek 14 Vizuální prezentace pro den .....           | 41 |
| Obrázek 15 Vizuální prezentace pro týden.....          | 42 |
| Obrázek 16 Vizuální prezentace pro měsíc .....         | 43 |
| Obrázek 17 Globální nastavení.....                     | 44 |
| Obrázek 18 Ukázka PDF .....                            | 45 |
| Obrázek 19 Blokový diagram algoritmu .....             | 48 |
| Obrázek 20 Blokový diagram algoritmu Importu 1/3 ..... | 51 |
| Obrázek 21 Blokový diagram algoritmu Importu 2/3 ..... | 51 |
| Obrázek 22 Blokový diagram algoritmu Importu 3/3 ..... | 51 |

## **SEZNAM ZKRATEK A ZNAČEK**

EPD – evidence pracovní doby

FPD – fond pracovní doby

D – dovolená

PC – pracovní cesta

N – nemoc

OČR – ošetřování člena rodiny

L – lékař

S – státní svátek

CSV – Comma-Separated Values

PDF – Portable Document Format

GUI – Graphical User Interface

UI – User Interface

STAG – Studijní agenda

OS – operační systém

DB – databáze

EF Core – Entity Framework Core

FK – Foreign Key

DI – Dependency Injection

IoC – Inversion of Control

ORM – Object–Relational Mapping

CLR – Common Language Runtime

XAML – eXtensible Application Markup Language

AXAML – Avalonia eXtensible Application Markup Language

MVC – Model–View–Controller

MVVM – Model–View–ViewModel

JIT – Just-In-Time

CI/CD – Continuous Integration / Continuous Deployment

## TERMINOLOGIE

**Avalonia UI** – Multiplatformní framework pro tvorbu uživatelských rozhraní v .NET s deklarativním popisem obrazovek v AXAML. Umožňuje jednotný vzhled a chování na Windows, Linuxu i macOS.

**Framework** – Sada knihoven, nástrojů a doporučených postupů, která dává aplikaci strukturu a opakovaně použitelné stavební bloky.

**Relační databáze** – Datový model založený na tabulkách a vztazích (vazby 1:1, 1:N, N:M) s důrazem na integritu a efektivní dotazování.

**MVVM** (Model–View–ViewModel) – Vzor, který odděluje data a logiku (Model), prezentační logiku (ViewModel) a zobrazení (View). Zlepšuje testovatelnost a udržitelnost UI.

**Uživatelské rozhraní (UI)** – Vizuální a interaktivní část aplikace (okna, prvky, šablony), v této práci vytvořená v Avalonia UI/AXAML.

**IDE** – Integrované vývojové prostředí (editor, debugger, správce závislostí a verzí) pro efektivní vývoj.

**VCS** – Systémy pro správu verzí (např. Git) pro sledování změn a týmovou spolupráci.

**STAG** – Studijní agenda, zdroj rozvrhových dat pro import.

**EPD** – Evidence pracovní doby, oficiální výkaz s měsíčním a týdenním součtem dle interních pravidel.

.

## ÚVOD

Evidence pracovní doby je běžná součást práce vyučujících na vysokých školách. Každý měsíc musí zaměstnanci správně zaznamenat, kolik hodin odpracovali, kdy měli pauzu a zda během měsíce chyběli kvůli dovolené, nemoci nebo jiným důvodům. I když to na první pohled vypadá jednoduše, realita bývá často složitější. Vyučující si vedou evidenci různě – pomocí nejednotných tabulek, ručním zapisováním nebo pomocí vlastních šablon. To často vede k chybám, nepřesnostem a zbytečnému času strávenému kontrolou.

Zvlášť náročné je to ve chvílích, kdy do běžného rozvrhu zasahují státní svátky, nepravidelnosti ve výuce nebo různé typy nepřítomnosti. V takových případech je těžké udržet správný přehled o tom, kolik hodin se skutečně odpracovalo a jak se to promítá do celkového měsíčního součtu. Mnoho vyučujících se navíc musí opakovaně vracet ke svým záznamům, aby vše opravili nebo přepočítali, protože výsledek neodpovídá předepsaným normám.

Tato práce se snaží tento problém vyřešit a nabídnout jednoduché a srozumitelné řešení. Cílem je vytvořit aplikaci, která převezme rozvrh vyučujícího a převede ho do podoby, která odpovídá pravidlům evidence pracovní doby. Uživatel už nebude muset počítat součty ručně ani přepisovat hodiny do tabulek. Aplikace načte data, zobrazí je v přehledné formě a umožní jednoduše upravit příchody, odchody, přestávky nebo zaznamenat absenci. Nakonec vygeneruje oficiální měsíční výkaz ve formátu PDF, který je připraven k odevzdání.

V teoretické části práce jsou vysvětleny základní principy, které s evidencí pracovní doby souvisí. Patří mezi ně například pracovní okno dne, očekávaný čas práce nebo způsoby, jak vyrovnat rozdíl mezi skutečně odpracovaným časem a tím, co předepisují normy. Také jsou popsány situace, kdy je potřeba brát ohled na státní svátky nebo jiné výjimky, aby byl výsledný výkaz správný a odpovídal pravidlům.

Praktická část se věnuje návrhu a tvorbě samotné aplikace. Popisuje, jak aplikace pracuje s daty, jak uživateli usnadňuje zadávání údajů a jak probíhá výpočet celkového času. Důraz je kladen na jednoduchost, srozumitelnost a spolehlivost při používání na různých zařízeních. Zvláštní pozornost je věnována tomu, aby výsledné výstupy byly přehledné a bezchybné.

Téma této práce je aktuální a důležité. Každý vyučující musí mít jasný a správný výkaz pracovní doby. Pokud systém evidence dobře funguje, šetří to čas, snižuje chybovost a zjednodušuje komunikaci mezi vyučujícím a administrativou. Výsledkem této práce je proto nejen funkční

aplikace, ale také ucelený pohled na to, jak by moderní a digitální evidence pracovní doby mohla fungovat jednoduše a spolehlivě.

# 1 TEORETICKÁ ČÁST

## 1.1 Evidence pracovní doby

Evidence pracovní doby je měsíční tabulka, ve které má každý kalendářní den vlastní řádek a kde zaznamenáváme přítomnost v práci i uznané důvody nepřítomnosti. Levá část tabulky zachycuje začátek směny, první a případně druhou přestávku a konec směny. Tyto údaje dohromady tvoří pracovní okno dne mezi příchodem a odchodem. V další části evidujeme odpracované hodiny, přesčas a neodpracované hodiny. Vpravo je poznámka s kódy událostí. Používáme zejména D pro dovolenou, N pro nemoc, OČR pro ošetřování člena rodiny, L pro lékaře, PC pro služební cestu a S pro svátek.

Abychom si rozuměli, opíráme se o tři pojmy. Odpracované hodiny jsou skutečně odvedená práce v daném dni uvnitř pracovního okna. Neodpracované hodiny jsou čas, který nebyl odpracován z uznaných důvodů, například dovolená, nemoc, návštěva lékaře nebo OČR. Přesčas je ta část práce, která po sečtení všech dnů v týdnu přesáhne týdenní fond. V praxi zapisujeme denní práci jako odpracované hodiny, uznané absence jako neodpracované hodiny a případné překročení týdenního fondu evidujeme jako přesčas.

Evidence vychází z týdenního fondu pracovní doby. Při plném úvazku jde o čtyřicet hodin. U zkráceného úvazku se fond řídí pracovní smlouvou. Případá-li na týden státní svátek nebo jiný den pracovního klidu, fond se za každý takový den snižuje o osm hodin. Na začátku a na konci měsíce se mohou objevit neúplné týdny. V takovém případě určujeme fond podle skutečného počtu pracovních dnů a každý z nich představuje osm hodin.

Denní zápis vychází z rozvrhu. Do tabulky přenášíme začátek směny, povinnou polední přestávku a konec směny, abychom přesně popsali pracovní okno. V typickém dni počítáme s osmi hodinami práce a třiceti minutami přestávky. Přestávku neevidujeme jako samostatné hodiny. Je zahrnutá v očekávaném čase uvnitř pracovního okna. Služební cesta v pracovním dni se chápe jako práce a zapisuje se podle skutečnosti až do osmi hodin. Práce vykonaná na služební cestě o víkendu se do evidence pracovních hodin nekládá.

Na konci týdne porovnáváme součet odpracovaných hodin s týdenním fondem. Pokud je součet vyšší, snažíme se přebytek rozprostřít do jiných pracovních dnů stejného týdne. Děláme to jen v rámci existujících pracovních oken těchto dnů. Nevytváříme nové přítomnosti mimo pracovní okna a nepřeléváme hodiny do jiných týdnů. Teprve když takové rozložení není možné, evidujeme rozdíl jako přesčas.

Pokud je součet nižší než fond, postupujeme opačně. Hledáme dny v témž týdnu, které mají volnou kapacitu v pracovním okně, a přeskládáme rozvrh práce tak, aby se deficit pokryl. Pokud deficit vznikl z uznané nepřítomnosti, například z dovolené nebo nemoci, zůstává zapsán jako neodpracované hodiny a týden považujeme za korektně vyrovnaný. Plné týdny chápeme jako uzavřené celky. Na začátku a na konci měsíce lze neúplné týdny posoudit společně, aby měsíční fond vyšel správně.

Správnost ověřujeme jednoduchou rovnicí, která odpovídá vzoru tabulky. Platí

$$\Sigma \text{odpracováno} - \text{přesčas} + \text{neodpracováno} = \text{měsíční fond (FPD)}.$$

Znamení mínus u položky přesčas používáme proto, že přesčas je již součástí odpracovaných hodin a touto úpravou se vyhneme dvojímu započtení. Souhrnný řádek Celkem na konci tabulky umožňuje provést kontrolu na první pohled.

Takto pojatá evidence i vyrovnávání zajišťují, že záznamy jsou čitelné, že každá úprava má jasné zdůvodnění a že opakované vyhodnocení nad stejnými daty dává stejný výsledek.

| Den    | Začátek<br>pracovní doby | 1. přestávka |       | 2. přestávka |    | Konec<br>pracovní<br>doby | Odpracováno | Přesčas  | Neodprac. | Poznámka<br>) |
|--------|--------------------------|--------------|-------|--------------|----|---------------------------|-------------|----------|-----------|---------------|
|        |                          | Od           | Do    | Od           | Do |                           |             |          |           |               |
| 1.     | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |               |
| 2.     |                          |              |       |              |    |                           |             |          |           |               |
| 3.     |                          |              |       |              |    |                           |             |          |           |               |
| 4.     | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |               |
| 5.     | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |               |
| 6.     | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |               |
| 7.     | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |               |
| 8.     | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |               |
| 9.     |                          |              |       |              |    |                           |             |          |           |               |
| 10.    |                          |              |       |              |    |                           |             |          |           |               |
| 11.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |               |
| 12.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |               |
| 13.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |               |
| 14.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |               |
| 15.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |               |
| 16.    |                          |              |       |              |    |                           |             |          |           |               |
| 17.    |                          |              |       |              |    |                           |             |          |           |               |
| 18.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |               |
| 19.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |               |
| 20.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |               |
| 21.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |               |
| 22.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |               |
| 23.    |                          |              |       |              |    |                           |             |          |           |               |
| 24.    |                          |              |       |              |    |                           |             |          |           |               |
| 25.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |               |
| 26.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |               |
| 27.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |               |
| 28.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |               |
| 29.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |               |
| 30.    |                          |              |       |              |    |                           |             |          |           |               |
| 31.    |                          |              |       |              |    |                           |             |          |           |               |
| Celkem |                          |              |       |              |    |                           | 168:00:00   | 00:00:00 | 00:00:00  |               |

kontr. č.  $\Sigma$  odprac. - přesčas + neodprac. hodin (odpovídá FPD): 168:00:00

Obrázek 1 Tabulka EPD

## 1.2 Architektura a vzory

Aplikace je desktopová a multiplatformní. Běží přímo na počítači uživatele, má přehledné grafické rozhraní, umí načíst rozvrh z CSV a vygenerovat oficiální PDF. Nepotřebuje žádný server ani skládání více síťových služeb. Díky tomu je nasazení jednoduché a práce s daty zůstává plně pod kontrolou uživatele.

Z těchto důvodů volím modulární monolit. Jde o jeden spustitelný program, uvnitř rozdělený do jasných částí: import CSV, vyrovnávací jádro, generátor PDF a uživatelské rozhraní.

Moduly spolu komunikují přes jednoduchá rozhraní a každý řeší svou úlohu. Tato volba snižuje provozní režii, zjednodušuje testování a urychluje vývoj. Zároveň odpadá složitost distribuované architektury, kde by se řešily verze služeb, síťové výpadky a bezpečnostní vrstvy [17][19].

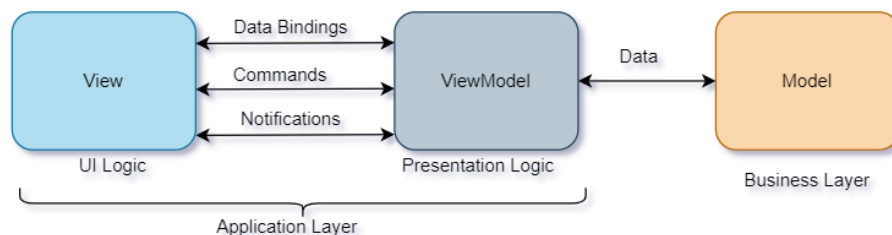
### **1.2.1 Co je architektura softwaru**

V této práci chápu architekturu softwaru jako soubor rozhodnutí, která dávají systému tvar i způsob chování. Určuje, z jakých částí se systém skládá, jaká mají rozhraní a jak si předávají data. Zároveň vymezuje odpovědnosti jednotlivých komponent a popisuje, jak dosáhnout požadovaných vlastností, jako je snadná změna, spolehlivost a bezpečnost. Takové pojetí odpovídá definicím Software Engineering Institute i běžné praxi v softwarovém inženýrství.[13]

### **1.2.2 MVVM – Model–View–ViewModel**

MVVM je vzor pro aplikace s deklarativním rozhraním, například v XAML nebo AXAML. Odděluje to, co uživatel vidí (View), od prezentační logiky (ViewModel) a od doménových dat a služeb (Model). Opírá se o datové vazby a příkazy. Když se změní stav ve ViewModelu, obrazovka se sama aktualizuje. Když uživatel klikne nebo odešle formulář, spustí se příkaz ve ViewModelu. Vztahy mezi částmi ukazuje Obrázek 1. Díky tomuto dělení je kód přehledný, má malou míru provázanosti a dobře se testuje [3][4].

V této práci používám MVVM takto. Obrazovky píšu v Avalonia UI (AXAML) a nechávám je čistě prezentační. Veškerá logika je ve třídách ViewModel, které lze testovat bez spuštění UI. Pro reaktivní stav a příkazy používám knihovnu ReactiveUI. Ta je navržena pro MVVM a podporuje deklarativní práci se stavem [4][12]. View komunikuje s ViewModelem pouze přes vazby a příkazy a k Modelu nepřistupuje přímo. ViewModel pracuje s Modelem přes rozhraní. Tím zůstává hranice mezi Model–View–ViewModel jasná a návrh je snadno udržitelný i testovatelný [3][4].

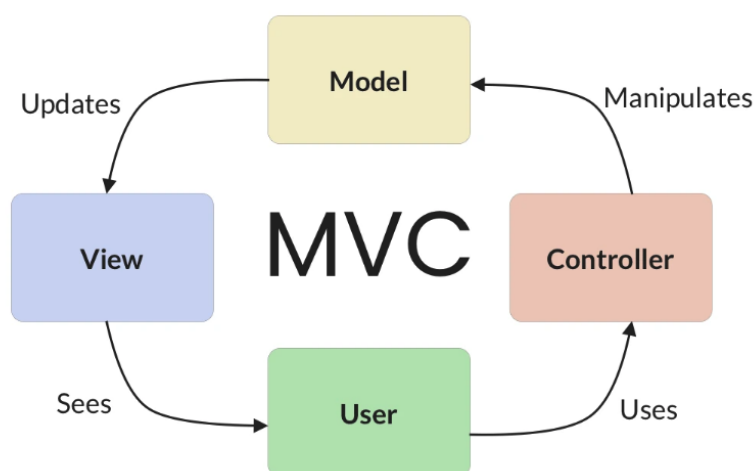


Obrázek 2: Architektura MVVM

### 1.2.3 MVC – Model–View–Controller

Pro úplnost stručně uvedu i architekturu MVC a proč v této práci volím MVVM. MVC dělí aplikaci na Model, View a Controller. Controller zpracuje vstup uživatele, rozhodne o akci a určí, který View se vykreslí. Ve webových aplikacích, například v ASP.NET MVC, jde o ověřený přístup s jasným oddělením odpovědností ve vrstvě prezentace [11].

U desktopových XAML aplikací (Avalonia, WPF, MAUI) je však přirozenější MVVM. XAML nativně podporuje datové vazby a příkazy, takže se změny ve ViewModelu automaticky promítají do View a uživatelské akce spouštějí logiku bez potřeby ručního kódu v Controlleru. MVVM se tak lépe hodí pro tento ekosystém, snižuje provázanost a zlepšuje testovatelnost i údržbu [3][11].



Obrázek 3 Architektura MVC

### 1.2.4 Monolit, modulární monolit a mikroslužby

Monolitická aplikace je jeden nasaditelný celek, tedy jeden proces nebo binárka. Uvnitř může mít vrstvy a knihovny, navenek se ale chová jako jedna jednotka. Je to srozumitelná volba pro menší a středně velké systémy a zároveň jednoduchá na provoz i nasazení.[17]

Modulární monolit je také jediný spustitelný program, uvnitř však drží přísně oddělené moduly s jasnými rozhraními. Typicky jde o části jako Import CSV, vyrovnávací jádro, generátor PDF a grafické rozhraní. Moduly spolu mluví pouze přes definovaná rozhraní a obcházení hranic se nepovoluje. Díky tomu je kód čitelnější, lépe se testuje a snadněji se udržuje. Pokud se projekt časem rozroste, lze vybrané moduly bez velkých změn vyčlenit do samostatných služeb.[17][19]

Mikroslužby dělí systém na více nezávisle nasaditelných částí. Každá služba má svůj životní cyklus a může se škálovat samostatně. Tento přístup ale přináší novou vrstvu složitosti. Je nutné řešit síťovou komunikaci, konzistenci dat, sledování a logování, nasazování více komponent i provozní automatizaci.[18][20]

Pro nový projekt je rozumné začít dobře navrženým modulárním monolitem. Dokud nejsou jasné důvody ke změně, těžíme z jednoduchosti a rychlého vývoje. Jakmile se objeví skutečné potřeby, například nezávislé škálování nebo pevné hranice týmů, je možné vybrané moduly převést na mikroslužby. Tím se vyhneme předčasné distribuované složitosti a platíme vyšší cenu až ve chvíli, kdy to dává smysl.[19]

## 1.3 Programovací jazyky

Tato kapitola vysvětluje, proč je pro tuto práci zvolen C# na platformě .NET. Tato kombinace umožňuje vytvářet plnohodnotnou desktopovou aplikaci pro Windows i Linux, přirozeně podporuje vzor MVVM a dobře spolupracuje s deklarativním UI. Díky datovým vazbám a příkazům se změny stavu promítají do obrazovek bez nadbytečného kódu a prezentační logiku lze pohodlně testovat i bez spuštěného rozhraní.[1][2][3][4]

Důležitá je i práce s daty a výstupy. Ekosystém .NET nabízí zralé knihovny pro CSV a spolehlivou integraci se SQLite, která slouží jako lokální úložiště bez potřeby serveru. Pro tiskové výstupy je k dispozici QuestPDF s „fluent“ API, které usnadňuje deterministické rozvržení tabulek a textu.[5][6]

Podíval jsem se také na rozumné alternativy. Electron s TypeScriptem láká moderním webovým UI a multiplatformností, ale přináší vyšší paměťové nároky a odlišný model práce s

UI. Python s Qt (PySide6) je rychlý pro prototypy, v této aplikaci však dávám přednost silné typové kontrole a návaznosti na .NET. Z prostředí JVM je zajímavý Kotlin Compose Multiplatform, jenže mobilní cíle zde neřeším a návaznost na knihovny .NET je pro tento projekt výhodnější.[7][8][9][10]

Shrnutí je prosté. C#/.NET spolu s Avalonia UI a ReactiveUI tvoří ucelený, testovatelný a multiplatformní základ. SQLite pokrývá lokální databázi srozumitelným SQL a QuestPDF spolehlivě generuje PDF. Tato volba splňuje požadavky práce na přenositelnost, podporu MVVM, práci s CSV/SQLite i kvalitní tiskové výstupy.[1][2][3][4][5][6]

### 1.3.1 Jazyk C#

C# je moderní, obecně použitelný jazyk s důrazem na produktivitu, čitelnost a bezpečnost kódu. Patří do otevřeného ekosystému .NET, který je zdarma a běží na Windows, Linuxu i macOS, takže jeden kód lze snadno přenášet mezi platformami.[1][2]

Pro uživatelské rozhraní používám Avalonia UI. Vychází z MVVM a datových vazeb, takže View zůstává čistě prezentační a logika je v testovatelných třídách ViewModel. Reaktivní práci se stavem a příkazy přirozeně podporuje knihovna ReactiveUI, která MVVM cílí přímo.[3][4]

Aplikace potřebuje rychle číst a zapisovat CSV, spolehlivě ukládat data lokálně a vytvářet PDF. SQLite do .NET zapadá bez problémů a QuestPDF nabízí deterministické rozvržení dokumentů v C#. Tato kombinace drží nízkou provozní složitost a šetří čas při vývoji.[5][6]

### 1.3.2 Jazyk SQL

V projektu používám SQL v kontextu lokální databáze SQLite. Data jsou uložena v tabulkách se sloupci a řádky a každý záznam je jednoznačně určen primárním klíčem. Vztahy mezi tabulkami vyjadřují cizí klíče, které lze v SQLite zapnout a kontrolovat, aby byla zajištěna referenční integrita.[5]

SQL pokrývá tři oblasti práce s daty. DDL definuje schéma (například příkazy CREATE TABLE a ALTER TABLE), DML mění obsah tabulek (INSERT, UPDATE, DELETE) a DQL vrací data pomocí SELECT. Vše se provádí v transakcích, které plní vlastnosti ACID. SQLite popisuje i režimy transakcí, které se hodí podle typu operace.[5]

Pro přehledy a výkazy jsou klíčové agregace a filtrování. SQLite podporuje GROUP BY, SUM, COUNT či AVG, a také CTE včetně rekurzivních dotazů a okenních funkcí pro pokročilé výpočty. Výkon zlepšují indexy na klíčích a často filtrovaných sloupcích.[5]

Je dobré rozumět tomu, jak SQLite zachází s daty. Používá tzv. manifest typing a pět tříd úložiště (INTEGER, REAL, TEXT, BLOB, NULL), přičemž sloupce mají typovou afinitu. Tento model je flexibilní, ale vyžaduje pečlivé zacházení s formáty, hlavně u dat a časů. V praxi pomůže ISO-8601, případně UNIX epoch, a vestavěné funkce jako date či strftime. V českém prostředí je vhodné myslet i na collation.[5]

Bezpečnost řeším parametrizovanými dotazy, případně ORM. V .NET lze pracovat přímo přes ADO.NET nebo použít Entity Framework Core s oficiálním providerem pro SQLite, což integraci ještě zjednoduší.[21]

Shrnutí: SQLite poskytuje stabilní a přenositelné úložiště pro výkazy a přehledy, omezení a transakce chrání integritu dat a indexy s agregacemi umožňují rychlé dotazy. V kombinaci s .NET je práce se SQL jednoduchá, bezpečná a dobře integrovaná do zbytku aplikace.[5][21]

## **1.4 Framework Avalonia UI**

Tato kapitola shrnuje základy Avalonia UI, otevřeného frameworku pro .NET. Avalonia vychází ze světa XAML a umožňuje deklarativně popsat uživatelské rozhraní, přirozeně podporuje architekturu MVVM a opírá se o datové vazby a příkazy. Dokumentace tento přístup doporučuje a popisuje, jak propojit View a ViewModel tak, aby se změny stavu promítaly do UI bez nadbytečného kódu [3].

Obrazovky jsou zapsané v souborech AXAML a zůstávají čistě prezentační. Logika a stav patří do tříd ViewModel, které vystavují vlastnosti a příkazy. Doménová data a služby tvoří Model. Tato separace snižuje provázanost, zjednodušuje testování a udržuje kód přehledný i při postupném rozšiřování aplikace [3].

Vazby propojují vlastnosti ViewModelu s prvky rozhraní a průběžně udržují jejich soulad. Příkazy zajišťují reakce na uživatelské akce bez psaní obsluh přímo do View. Díky tomu zůstává prezentace deklarativní a odpadá velká část „lepidlového“ kódu, který bývá náchylný k chybám [3].

Pro reaktivní práci se stavem používám ReactiveUI. Knihovna poskytuje reaktivní vlastnosti, příkazy i pozorovatelné toky a dobře zapadá do MVVM. ViewModely lze díky tomu testovat bez spuštění UI, a přitom zachovat čitelnost a jednoznačné datové toky [4][12].

Opakovatelné interakce je možné přidávat pomocí konceptu Behaviors. Ten je známý z WPF a popsáný v materiálech Microsoftu. V Avalonia UI je dostupná knihovna Avalonia.Xaml.Behaviors spolu s příklady použití, například spouštění příkazů na konkrétní

události nebo podmíněné chování. Behaviors pomáhají držet View jednoduché a znovupoužitelné, protože běžné interakce lze skládat bez zásahu do logiky ViewModelu [14][15][16].

Stejně jako jiné XAML frameworky nabízí Avalonia styly, šablony ovládacích prvků a sdílené zdroje. Umožňují nastavit vzhled konzistentně napříč obrazovkami a oddělit vizuální vrstvu od chování, což je v souladu s MVVM a usnadňuje dlouhodobou údržbu [3].

Souhrnně řečeno, Avalonia UI přináší deklarativní GUI, čisté oddělení vzhledu a logiky, možnost reaktivní implementace s ReactiveUI a rozšiřitelnost přes Behaviors. Tyto vlastnosti jsou pro tuto práci klíčové, protože usnadňují návrh přehledů EPD, validací a stavové logiky a současně drží kód jednoduchý a dobře testovatelný [3][4][12][15][16].

## **1.5 Vývojové prostředí**

Tato kapitola popisuje nástroje, na kterých projekt stojí, a proč byly zvoleny. Cílem je, aby šel zdrojový kód snadno otevřít, spustit a rozvíjet i na jiném počítači a s odstupem času. Pro vývoj v C#/ .NET používám integrované prostředí od Microsoftu a pro lokální databázi samostatný nástroj nad SQLite. Díky tomu je celý řetězec od kódu po data jednoduchý a opakovatelný.[2]

### **1.5.1 Microsoft Visual Studio**

Microsoft Visual Studio je hlavní IDE projektu. V jednom prostředí píšu kód, spravuji balíčky přes NuGet, spouštím aplikaci a ladím chyby. Pomáhá IntelliSense s nápovědou a doplňováním, refaktoring, vestavěný debugger i profilace výkonu. Vše je zaměřené na to, aby šla změna rychle ověřit a bezpečně začlenit do hlavní větve.[22]

Součástí IDE je integrace s Git a GitHubem i s Azure DevOps. Historii změn, větvení a slučování tak lze dělat přímo z editoru. Pro párové programování a rychlé konzultace je k dispozici Live Share, který umožní sdílet sezení v reálném čase.[22]

Pro návrh uživatelského rozhraní využívám Avalonia s oficiálním rozšířením pro Visual Studio. Náhledy AXAML, šablony a napojení na MVVM výrazně urychlují práci. Balíčky do projektu přidávám přes NuGet, takže jsou verze knihoven jasně dané a opakovatelné. Na Windows běží Visual Studio, na Linuxu lze stejný kód otevřít ve Visual Studio Code nebo pracovat přes .NET CLI.[3][22]

### **1.5.2 SQLiteStudio**

SQLiteStudio je grafický nástroj pro práci se soubory SQLite. Umožňuje prohlížet a upravovat tabulky a indexy, spouštět SQL dotazy se zvýrazněním syntaxe a mít po ruce historii příkazů.

Hodí se pro rychlou kontrolu schématu i pro přípravu testovacích dat mimo běžící aplikaci.[5][25]

Podporuje import a export dat, například z a do CSV, a umí zobrazit vztahy mezi tabulkami v jednoduchém ER náhledu. Díky tomu lze rychle ověřit, že omezení a klíče odpovídají očekávání, ještě, než se změny promítnou do aplikace.[25]

## 1.6 Knihovny

Knihovny jsou znovupoužitelné části softwaru, které přinášejí hotové funkce, třídy a vzory pro řešení opakujících se úloh. Díky nim není nutné psát vše od nuly. Umožňují soustředit se na doménový problém a přenechat běžnou infrastrukturu prověřenému kódu. V praxi to vede k rychlejšímu vývoji, menšímu počtu chyb a stabilnějšímu výsledku.

V .NET světě se knihovny spravují přes NuGet. Projekt jen uvede, které balíčky potřebuje, a nástroje při buildu automaticky stáhnou správné verze včetně transitivních závislostí. Stejně nastavení tak funguje na různých počítačích i v CI. Vývojář má k dispozici verzování, uzamčení závislostí a jednoduchou aktualizaci na novější vydání. Výsledkem je opakovatelné prostředí a menší riziko „funguje to jen u mě“ situací [22].

Teoretické pozadí práce s knihovnami stojí na několika zásadách. Důležitá je stabilita veřejného API, aby aktualizace nevyžadovala rozsáhlé zásahy do kódu. Hodí se malé a jasně vymezené závislosti, které dělají jednu věc dobře, místo těžkých vše-v-jednom balíčků. Záleží také na licencích a na tom, zda je projekt aktivně udržovaný a má průběžná vydání. V bezpečnosti pomáhá pravidelná kontrola známých zranitelností a rozumné tempo aktualizací. Při návrhu je vhodné třetí strany zapouzdřit za rozhraní, aby bylo možné je v budoucnu vyměnit, a v testech používat falešné implementace místo skutečných balíčků. Tím se snižuje provázanost a roste testovatelnost.

Tento projekt využívá knihovny, které zapadají do zvolené architektury MVVM a podporují multiplatformní běh. Avalonia UI poskytuje deklarativní vrstvu zobrazení a dobře si rozumí s datovými vazbami. ReactiveUI přidává reaktivní práci se stavem a příkazy. Pro práci s lokální databází slouží SQLite a oficiální .NET integrace. Tiskové výstupy zajišťuje QuestPDF, který dovoluje přesně ovládnout rozvržení tabulek a textu. Díky těmto stavebním blokům je možné mít čisté oddělení zodpovědností, jednoduše testovat prezentační logiku a spolehlivě generovat výstupy, které EPD potřebuje [3][4][5][6][21].

NuGet v této skladbě funguje jako jednotné místo pravdy pro závislosti. Uzamkne verze, zajistí opakovatelný build a usnadní přechod na novější vydání jednotlivých knihoven. To zkracuje cyklus změna-build-test a zajišťuje, že projekt půjde bez potíží otevřít a sestavit i s odstupem času nebo na jiném počítači [22].

## 2 PRAKTICKÁ ČÁST

Tato část popisuje, jak je aplikace skutečně postavená a jak spolu jednotlivé části pracují. Zaměřuji se na použité jazyky, strukturu dat, hlavní služby, uživatelské rozhraní a tok operací od importu až po export PDF.

### 2.1 Programovací jazyky

V projektu kombinuji C# na platformě .NET pro aplikační logiku a AXAML pro deklarativní popis obrazovek v Avalonia UI. C# zajišťuje práci s daty, algoritmy a integrace jako import CSV, přístup do SQLite a generování PDF. AXAML drží prezentační vrstvu, v níž jsou definované obrazovky a vazby na MVVM. Díky tomu zůstává logika oddělená od vzhledu, ale přes data binding a příkazy je propojení přímé a bez zbytečného lepidlového kódu.

#### 2.1.1 Jazyk C#

C# tvoří páteř řešení a přes Entity Framework Core mapuje modely na SQLite. Schéma databáze odpovídá tomu v kódu a je řízené migracemi, takže změny jsou sledovatelné a opakovatelné.

**Jádrová tabulka Events.** Všechny denní události jsou sjednocené v jednom místě. Do stejné tabulky ukládám výuku, běžnou práci, dny pracovního klidu, dovolenou, nemoc, OČR, návštěvu lékaře i služební cesty. Základ tvoří časové údaje *StartTime*, *EndTime* a příznak *AllDay*. Typ a původ události nesou pole *EventType*, *IsAutoGenerated*, *ParentEventId* a metainformace o importu *ImportBatchId*, *ImportLabel*. Pro pracovní okno ukládám *ArrivalTime*, *DepartureTime*, *LunchStart*, *LunchEnd*. Kvalitu dat hlídá příznak *HasCollision*, který označí časové překryvy.

**Nastavení.** Tabulka *GlobalSettings* nese výchozí pravidla a limity pro období. Obsahuje přednastavené příchody a odchody podle dnů v týdnu, parametry oběda včetně minimální a maximální délky a názvy automatických bloků před a po přestávce. *UserSettings* umožňuje pro konkrétní datum tato pravidla přenastavit. Historii změn schématu ukládá systémová tabulka *\_\_EFMigrationsHistory*.

Služby a pracovní tok.

- **Import rozvrhu (CSV):** služba načte dávku, určí typy událostí a uloží je do Events. Rozvrhové bloky se nepřepisují a slouží jako referenční data.

- **Vyrovňávání a validace:** motor uplatní pravidla evidence, vygeneruje podpůrné bloky přestávky, jemně upraví okraje automatických bloků v povolených mezích a označí kolize. Výsledek je konzistentní rozpis dne, týdne a měsíce.
- **Export PDF (QuestPDF):** generuje měsíční tabulku s hlavičkou, součty a zvýrazněním svátků a celodenních stavů. Rozvržení je deterministické, takže stejné vstupy produkují stejné výstupy.
- **ViewModely:** zajišťují filtrování a náhled, validaci, spouštění importu a exportu a komunikaci s UI přes vazby a příkazy.

Aplikace používá `async/await`, aby zpracování souborů, databázové operace a tvorba PDF neblokovaly rozhraní. Pro výpočty a agregace využívám LINQ. Při ukládání i úpravách probíhá kontrola konzistence: bloky v jednom dni se nesmějí nelogicky překrývat, celodenní události se zapisují jedním záznamem s `AllDay = true`, přestávky respektují meze a týdenní i měsíční součty odpovídají požadavkům. Silné typování v C# odhalí mnoho chyb už při kompilaci a jednotná tabulka `Events` výrazně zjednodušuje dotazy i generování výstupů. Rozdělení na modely, služby a ViewModely drží kód přehledný a připravený na rozšíření.

### 2.1.2 Jazyk XAML

AXAML popisuje obrazovky v Avalonia UI. Patří sem průvodce importem CSV, kalendářový přehled, editor denních záznamů a náhled před exportem PDF. View neobsahuje aplikační logiku. Tu drží ViewModely v C#, které vystavují vlastnosti a příkazy.

Vazby propojují prvky rozhraní s daty a agregacemi ve ViewModelech, například kolekci měsíčních událostí nebo přepočtené týdenní a měsíční součty. Uživatelské akce spouštějí příkazy typu `GenerateEPDCommand`, `CreateEventCommand` nebo `ShowPdfPreview`. Změny stavu se projeví okamžitě, bez ručního překreslování.

Opakující se části rozhraní jsou definované pomocí `DataTemplate` a `ControlTemplate`, takže řádky, denní buňky, značky svátků a nepřítomností jsou na jednom místě a snadno se udržují. Styly sjednocují barvy, písma a odsazení, což umožňuje měnit vzhled bez zásahu do C#.

Ve spojení s ReactiveUI jsou ViewModely reaktivní. Změna dat, například vložení půldenní dovolené, okamžitě přepočte součty a aktualizuje náhled tabulky. Rozhraní tak zůstává plynulé i při práci s většími měsíci. AXAML drží prezentační vrstvu izolovanou a současně pevně napojenou přes bindingy a příkazy, takže lze UI a logiku vyvíjet nezávisle a rychle nad nimi iterovat.

## 2.2 Struktura aplikace

Aplikace je postavená na vzoru MVVM, který v Avalonia UI funguje přirozeně. Prezentační vrstva je oddělená od logiky a dat, takže se části nemíchají a změny se dělají bez vedlejších efektů. Díky tomu jde projekt snadno rozšiřovat, testovat i udržovat.

**Model a databáze.** Perzistentní data ukládám přes Entity Framework Core do SQLite. Základ tvoří třídy *Event*, *UserSettings* a *GlobalSettings*.

- *Event* nese informace o denních událostech (výuka, práce, dovolená, nemoc, OČR, lékař, služební cesta) včetně času začátku a konce, typu a podpůrných údajů pro pracovní okno a přestávku.
  - *UserSettings* drží individuální nastavení pro konkrétní den (příchod, odchod, oběd).
  - *GlobalSettings* obsahuje výchozí pracovní časy, parametry přestávek a identifikační údaje vyučujícího.
- Datový přístup zajišťuje *AppDbContext*, který definuje tabulky i mapování vztahů a řeší ukládání časů.

View (AXAML). Vzhled tvoří .axaml soubory: *MainWindow.axaml* s navigací a kalendářem, pohledy *DayView.axaml*, *WeekView.axaml*, *MonthView.axaml*, dialogy jako *CreateEventDialog.axaml* a *GlobalUserSettingsView.axaml*. Pro interakce používám Behaviors (např. *ShowCollisionMessageBehavior* pro upozornění na kolize). Levý panel hlavního okna přepíná den/týden/měsíc, otevírá nastavení a export PDF. Pravý panel zobrazuje data kalendáře.

ViewModely. Hlavní *MainWindowViewModel* načítá data a přepíná režimy zobrazení. *DayViewModel*, *WeekViewModel* a *MonthViewModel* řeší logiku pro svoje pohledy. *GlobalUserSettingsViewModel* spravuje pracovní intervaly a přestávky. *CreateEventDialogViewModel* validuje vstupy při tvorbě a úpravě událostí. *PdfPreviewViewModel* připravuje podklady pro export a kontrolu měsíčních součtů. Všechny změny se do UI propisují přes data binding, takže uživatel vidí aktuální stav hned.

Služby. Aplicační logika běží ve službách mimo ViewModely.

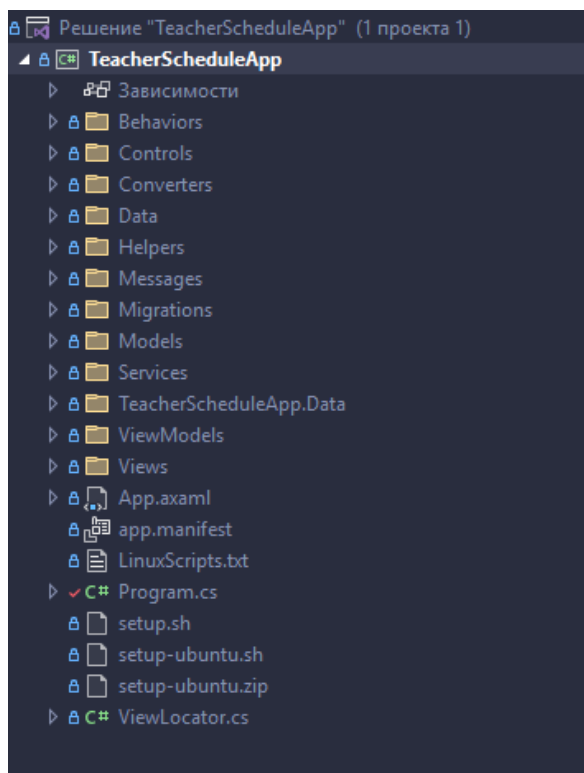
- *EventService* dělá CRUD nad událostmi a provádí vyrovnávání pracovních hodin mezi dny podle nastavených pravidel.
- *WorkingHoursCalculatorService* počítá odpracovaný čas a součty.

- *AutomaticEventsGeneratorService* vytváří výchozí bloky na základě globálního nastavení a rozvrhu.
- *PdfService* generuje měsíční tabulky do PDF.
- *EPDGenerator* načítá rozvrh ze souboru CSV (IS STAG) a převádí jej na interní události.
- *GlobalSettingsService* a *SettingsService* ukládají a načítají uživatelská i globální nastavení.

Tok práce. Uživatel naimportuje CSV s rozvrhem, aplikace naplní Events, provede vyrovnání podle pravidel, zobrazí výsledek v kalendáři a na závěr vygeneruje oficiální PDF. Díky rozdělení na model, ViewModely a služby zůstává kód přehledný a změny v jedné vrstvě nezasahují do ostatních.

## 2.4 Souborová struktura aplikace

Projekt je rozdělený do složek tak, aby byly jasně oddělené vrstvy i funkce. Kód pro databázi, prezentační logiku, uživatelské rozhraní a podpůrné utility má své místo. Díky tomu se v projektu rychle orientuje jeden vývojář i celý tým a změny se dělají bez zbytečných zásahů do ostatních částí.



Obrázek 4: Souborová struktura aplikace

Toto členění drží odpovědnosti přehledně od sebe, usnadňuje rozšíření o nové obrazovky nebo služby a zkracuje čas potřebný na úpravy i ladění.

#### **2.4.1 Složka Behaviors**

Obsahuje třídy z knihovny Avalonia.Xaml.Behaviors. Přidávají interakce k prvkům UI bez úprav jejich šablon. Typické příklady jsou reakce na kliknutí v kalendáři, zavírání dialogů s potvrzením akce a zobrazení chybových nebo kolizních hlášení. Díky tomuto přístupu zůstává View jednoduché a chování lze snadno znovu použít.

#### **2.4.2 Složka Controls**

Sdružuje vlastní ovládací prvky nad rámec základních komponent Avalonie. Slouží k vykreslení událostí v kalendáři i k rozvržení panelů. Opakovaným použitím drží aplikace konzistentní vzhled a snižuje se množství kódu v jednotlivých View.

#### **2.4.3 Složka Converters**

Obsahuje převodníky implementující *IValueConverter*. Ty převádějí data z ViewModelů do podoby vhodné pro UI. Jde například o mapování logických hodnot na viditelnost či barvu, výpočet pozic a délek bloků v kalendáři nebo převod času na grafické rozměry. Logika transformace je mimo View, což pomáhá udržet čisté MVVM.

#### **2.4.4 Složka Data**

Zde je datový kontext *AppDbContext*. Definuje přístup k databázi přes Entity Framework Core, entity pro události a nastavení a připojení k souboru SQLite v uživatelském profilu. Řeší mapování vlastností na sloupce a sjednocuje ukládání časových hodnot pomocí konvertorů.

```

Ссылка: 42 | Egor Razboinikov, 3 дн. назад | Автор: 1, изменений: 3
public class AppDbContext : DbContext
{
    Ссылка: 45 | Egor Razboinikov, 112 дн. назад | Автор: 1, изменение: 1
    public DbSet<Event> Events { get; set; }
    Ссылка: 6 | Egor Razboinikov, 112 дн. назад | Автор: 1, изменение: 1
    public DbSet<UserSettings> UserSettings { get; set; }
    Ссылка: 4 | Egor Razboinikov, 112 дн. назад | Автор: 1, изменение: 1
    public DbSet<GlobalSettings> GlobalSettings { get; set; }

    Ссылка: 0 | Egor Razboinikov, 112 дн. назад | Автор: 1, изменение: 1
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        string baseDir;
        if (RuntimeInformation.IsOSPlatform(OSPlatform.Windows))
            baseDir = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData);
        else
            baseDir = Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.UserProfile), ".config");

        var appDir = Path.Combine(baseDir, "TeacherScheduleApp");
        Directory.CreateDirectory(appDir);

        var dbPath = Path.Combine(appDir, "teacherapp.db");
        optionsBuilder.UseSqlite($"Data Source={dbPath}");
    }

    Ссылка: 0 | Egor Razboinikov, 3 дн. назад | Автор: 1, изменений: 3
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        var dateTimeConverter = new ValueConverter<DateTime, string>(
            v => v.ToString("yyyy-MM-dd HH:mm:ss", CultureInfo.InvariantCulture),
            v => DateTime.ParseExact(v, "yyyy-MM-dd HH:mm:ss", CultureInfo.InvariantCulture)
        );

        modelBuilder.Entity<Event>(b =>
        {
            b.HasKey(e => e.Id);
            b.Property(e => e.Id).ValueGeneratedOnAdd();
        });
        modelBuilder.Entity<Event>()
            .HasOne<Event>()
            .WithMany()
            .HasForeignKey(e => e.ParentEventId)
            .IsRequired(false)
            .OnDelete(DeleteBehavior.Cascade);

        modelBuilder.Entity<Event>()
            .Property(e => e.StartTime)
            .HasConversion(dateTimeConverter);

        modelBuilder.Entity<Event>()
            .Property(e => e.EndTime)
            .HasConversion(dateTimeConverter);
        base.OnModelCreating(modelBuilder);
    }
}

```

Obrázek 5 Ukázka kódu třídy AppDbContext

## 2.4.5 Složka Helpers

Obsahuje drobné pomocné třídy používané napříč projektem. Patří sem získání reference na hlavní okno, práce se soubory a poskytovatel státních svátků postavený na knihovně *Nager.Date*. Utility jsou malé, samostatné a snadno testovatelné.

## 2.4.6 Složka Messages

Definuje zprávy pro komunikaci mezi částmi systému bez přímých závislostí. ViewModely tímto kanálem dostávají informaci o vytvoření automatických událostí, dokončení generování PDF nebo změně nastavení. Moduly si tak nepředávají reference a zůstávají volně svázané.

### 2.4.7 Složka Models

Obsahuje datové entity mapované pomocí Entity Frameworku na tabulky v SQLite. Najdete zde třídy pro události, globální i uživatelská nastavení a podpůrné struktury pro výuku. Jde o jednoduché POCO objekty bez prezentační logiky.

### 2.4.8 Složka Services

Zde žije aplikační logika. *EventService* spravuje události a provádí vyrovnávání pracovních hodin mezi dny. *WorkingHoursCalculatorService* počítá odpracovaný čas a součty. *AutomaticEventsGeneratorService* vytváří automatické bloky podle globálních pravidel a rozvrhu. *PdfService* generuje měsíční přehled do PDF. *EPDGenerator* načítá rozvrh vyučujícího ze souboru CSV (IS STAG) a převádí jej na interní události.

### 2.4.9 Složka ViewModels

Soustřeďuje prezentační logiku. *MainWindowViewModel* načítá data a přepíná režimy zobrazení. *DayViewModel*, *WeekViewModel* a *MonthViewModel* řeší práci s denním, týdenním a měsíčním pohledem. *GlobalUserSettingsViewModel* spravuje pracovní intervaly a přestávky. *CreateEventDialogViewModel* validuje vstupy při tvorbě a editaci událostí. *PdfPreviewViewModel* připravuje data pro náhled a export. Změny se do UI propisují přes data binding.

### 2.4.10 Složka Views

Tvoří vizuální vrstvu aplikace. Obsahuje .axaml soubory a minimální code-behind pro hlavní okno, kalendářové pohledy a dialogy. Úpravy vzhledu se dělají zde, logika zůstává ve ViewModelu, takže se obě části vyvíjejí nezávisle.

### 2.4.11 Složka Kořenové soubory projektu

V kořeni jsou globální zdroje a start aplikace. *App.axaml* definuje styly a šablony, jeho code-behind inicializuje hlavní okno a životní cyklus. *Program.cs* představuje vstupní bod. Manifest nese metadata pro Windows. *ViewLocator* propojuje ViewModely s odpovídajícími Views. Struktura je připravená na rozšíření a změny bez zásahů do ostatních částí projektu.

## 2.5 Struktura databáze

Aplikace ukládá data do jednoho souboru SQLite a používá Entity Framework Core pro mapování entit i migrace. Schéma je záměrně jednoduché a bez relačních vazeb na úrovni databáze. Všechno, co se týká pravidel a kontroly konzistence, řeší aplikace při ukládání a zpracování dat.

## 2.5.1 Přehled tabulek

### Events — popis sloupců (k Obrázku 7)

Tabulka obsahuje všechny denní události, které zadá uživatel nebo je vytvoří aplikace; systémové záznamy pro víkendy a svátky se neukládají. *Id* (INTEGER) je primární klíč. *StartTime* a *EndTime* (TEXT) vymezují časový úsek události. *EventType* (INTEGER) je kód typu události, např. práce, dovolená, nemoc, OČR, lékař nebo služební cesta. *Title* a *Description* (TEXT) nesou volitelné popisky. *AllDay* (INTEGER) označuje celodenní stav. *ArrivalTime*, *DepartureTime*, *LunchStart* a *LunchEnd* (TEXT) drží skutečné okraje dne a polední přestávku pro daný den. *IsAutoGenerated* (INTEGER) říká, že blok vznikl automaticky, typicky jako dopočtená přestávka nebo okraj dne. *ParentEventId* (INTEGER) je logická návaznost na zdrojovou událost (bez databázového FK). *HasCollision* (INTEGER) značí, že byl zjištěn časový překryv dvou událostí. *ImportBatchId* a *ImportLabel* (TEXT) identifikují šarži a původ importu. *AutoGeneratedForDate* (TEXT) uvádí datum, pro které byl automatický blok vytvořen. *IsDeleted* (INTEGER) slouží pro měkké smazání, takže záznam zůstává v historii, ale běžné dotazy ho neberou v potaz.

|                      |         |                                                                                       |                                                                                       |
|----------------------|---------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Id                   | INTEGER |  |                                                                                       |
| AllDay               | INTEGER |                                                                                       |                                                                                       |
| ArrivalTime          | TEXT    |                                                                                       |                                                                                       |
| AutoGeneratedForDate | TEXT    |                                                                                       |                                                                                       |
| DepartureTime        | TEXT    |                                                                                       |                                                                                       |
| Description          | TEXT    |                                                                                       |                                                                                       |
| EndTime              | TEXT    |                                                                                       |                                                                                       |
| EventType            | INTEGER |                                                                                       |                                                                                       |
| HasCollision         | INTEGER |                                                                                       |                                                                                       |
| IsAutoGenerated      | INTEGER |                                                                                       |                                                                                       |
| IsDeleted            | INTEGER |                                                                                       |                                                                                       |
| LunchEnd             | TEXT    |                                                                                       |                                                                                       |
| LunchStart           | TEXT    |                                                                                       |                                                                                       |
| ParentEventId        | INTEGER |                                                                                       |  |
| StartTime            | TEXT    |                                                                                       |                                                                                       |
| Title                | TEXT    |                                                                                       |                                                                                       |
| ImportBatchId        | TEXT    |                                                                                       |                                                                                       |
| ImportLabel          | TEXT    |                                                                                       |                                                                                       |

Obrázek 6 Struktura tabulky Events

## GlobalSettings

Výchozí pravidla a hodnoty pro období: identifikace vyučujícího, globální okno dne, denní defaulty (příchod, odchod, oběd), limity délky přestávky a názvy automatických bloků.

|                       |         |                                                                                     |
|-----------------------|---------|-------------------------------------------------------------------------------------|
| Id                    | INTEGER |  |
| Year                  | INTEGER |                                                                                     |
| GlobalStartTime       | TEXT    |                                                                                     |
| GlobalEndTime         | TEXT    |                                                                                     |
| EmployeeName          | TEXT    |                                                                                     |
| Department            | TEXT    |                                                                                     |
| MondayArrival         | TEXT    |                                                                                     |
| MondayDeparture       | TEXT    |                                                                                     |
| MondayLunchStart      | TEXT    |                                                                                     |
| MondayLunchEnd        | TEXT    |                                                                                     |
| TuesdayArrival        | TEXT    |                                                                                     |
| TuesdayDeparture      | TEXT    |                                                                                     |
| TuesdayLunchStart     | TEXT    |                                                                                     |
| TuesdayLunchEnd       | TEXT    |                                                                                     |
| WednesdayArrival      | TEXT    |                                                                                     |
| WednesdayDeparture    | TEXT    |                                                                                     |
| WednesdayLunchStart   | TEXT    |                                                                                     |
| WednesdayLunchEnd     | TEXT    |                                                                                     |
| ThursdayArrival       | TEXT    |                                                                                     |
| ThursdayDeparture     | TEXT    |                                                                                     |
| ThursdayLunchStart    | TEXT    |                                                                                     |
| ThursdayLunchEnd      | TEXT    |                                                                                     |
| FridayArrival         | TEXT    |                                                                                     |
| FridayDeparture       | TEXT    |                                                                                     |
| FridayLunchStart      | TEXT    |                                                                                     |
| FridayLunchEnd        | TEXT    |                                                                                     |
| MinBreakDuration      | TEXT    |                                                                                     |
| MaxBreakDuration      | TEXT    |                                                                                     |
| AutoEventNamePreLunch | TEXT    |                                                                                     |
| AutoEventNameLunch    | TEXT    |                                                                                     |

Obrázek 7 Struktura tabulky GlobalSettings

## UserSettings

Přenastavení pro konkrétní datum: *ArrivalTime*, *DepartureTime*, *LunchStart*, *LunchEnd*. Pokud záznam existuje, přebije hodnoty z GlobalSettings.

|               |         |                                                                                     |
|---------------|---------|-------------------------------------------------------------------------------------|
| Id            | INTEGER |  |
| Date          | TEXT    |                                                                                     |
| ArrivalTime   | TEXT    |                                                                                     |
| DepartureTime | TEXT    |                                                                                     |
| LunchStart    | TEXT    |                                                                                     |
| LunchEnd      | TEXT    |                                                                                     |

Obrázek 8 Struktura tabulky *UserSettings*

## EFMigrationsHistory

Technická tabulka EF Core s historií migrací.

Zvolili jsme jednoduchost a odolnost souboru. SQLite je souborová databáze a lehká struktura usnadňuje zálohování, kopírování mezi stroji i rychlé zásahy při testování. Import probíhá po dávkách a dočasně se mohou objevit „čekající“ záznamy, které se doplní až po validaci. Bez cizích klíčů import nepadá na pořadí insertů a je snazší dorovnání. Pravidla a invarianty hlídají služby aplikace, takže je můžeme verzovat v kódu a nemusíme udržovat databázové triggery. Typické dotazy filtrují podle času a šarží, takže nám víc pomohou indexy nad časem než formální referenční omezení.

### 2.5.2 Shrnutí rozhodnutí

Databáze bez FK a bez uložených víkendů či svátků je menší, přenositelnější a tolerantní k dávkovému importu. Integritu drží aplikační logika: validace při importu a úpravách, jednotná časová mřížka, pravidla EPD a označování kolizí. Jednotná tabulka *Events* zjednodušuje dotazování i export. *GlobalSettings* a *UserSettings* pokrývají model „výchozí hodnoty vs. výjimky“ bez nutnosti relačních vazeb.

## 2.6 Uživatelské rozhraní aplikace

Rozhraní je navrženo jednoduše a přímočaře. Vše důležité je na dosah, obrazovky mají jednotnou logiku a prvky se chovají stejně napříč celou aplikací. Díky tomu se uživatel rychle zorientuje a může se soustředit na práci, ne na ovládání.

### 2.6.1 Hlavní stránka aplikace

Po spuštění se zobrazí hlavní okno rozdělené na dva panely. Vlevo jsou ovládací prvky, vpravo zvolený pohled kalendáře. Při prvním startu se také připraví výchozí data pro aktuální období. Výchozí zobrazení je dnešní den.

Levý panel obsahuje čtyři tlačítka:

- **Vytvořit událost** otevře dialog pro nový záznam.
- **Generovat EPD** spustí import rozvrhu ze souboru CSV ze STAGu.
- **Globální nastavení** otevře okno s výchozími časy a údaji.
- **Příklad PDF** ukáže náhled výstupu.

Pod tlačítka je kalendář pro rychlý výběr dne. Následuje přehled nahraných souborů a přepínač pohledu Den / Týden / Měsíc. V denním režimu se zobrazí blok nastavení dne se čtyřmi poli (příchod, odchod, začátek a konec polední pauzy) a tlačítkem Uložit. Úplně dole je souhrn, kolik hodin je odpracováno vůči normě.

Vytvořit událost

Načíst rozvrh

Globální nastavení

EPD náhled

srpen 2025

P

Ú

S

Č

P

S

N

|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| 28 | 29 | 30 | 31 | 1  | 2  | 3  |
| 4  | 5  | 6  | 7  | 8  | 9  | 10 |
| 11 | 12 | 13 | 14 | 15 | 16 | 17 |
| 18 | 19 | 20 | 21 | 22 | 23 | 24 |
| 25 | 26 | 27 | 28 | 29 | 30 | 31 |
| 1  | 2  | 3  | 4  | 5  | 6  | 7  |

Načtený soubory

Rozvrh

Nastavení pro den:

25.08.2025

Příchod:

08:30

Odchod:

17:00

Oběd od:

12:30

Oběd do:

13:00

Uložit nastavení

Obrázek 9 Levý panel aplikace

| ← Den: 25 srp 2025 → Dnes |                                                                 |
|---------------------------|-----------------------------------------------------------------|
| 04:00                     |                                                                 |
| 05:00                     |                                                                 |
| 06:00                     |                                                                 |
| 07:00                     |                                                                 |
| 08:00                     | <input checked="" type="radio"/> <b>Dopolední pracovní doba</b> |
| 09:00                     |                                                                 |
| 10:00                     |                                                                 |
| 11:00                     |                                                                 |
| 12:00                     |                                                                 |
| 13:00                     | <input checked="" type="radio"/> <b>Oběd</b>                    |
| 14:00                     | <input checked="" type="radio"/> <b>Odpolední pracovní doba</b> |
| 15:00                     |                                                                 |
| 16:00                     |                                                                 |
| 17:00                     |                                                                 |
| 18:00                     |                                                                 |

Obrázek 10 Pravý panel aplikace

### 2.6.2 Dialogové okno pro vytvoření události

Dialogové okno obsahuje následující prvky. První pole slouží k zadání nebo úpravě názvu události. Toto pole je povinné, při pokusu o vytvoření události bez názvu se zobrazí dialogové okno s chybou. Druhé pole již není povinné a slouží k přidání popisu události. Poté následuje tlačítko, které určuje, zda bude událost trvat celý den. Poté následuje blok pro nastavení data a času události. Za blokem pro nastavení data je prvek, který umožňuje vybrat typ události (pracovní, volno, služební cesta atd.). A posledními prvky jsou tlačítko pro zrušení, které zavře okno, a tlačítko pro uložení, které událost vytvoří a okno zavře.

**Nová událost**

**Název**

**Popis**

☐ Celý den

**Začátek:**

4 květen 2025 00:00

**Konec:**

4 květen 2025 00:00

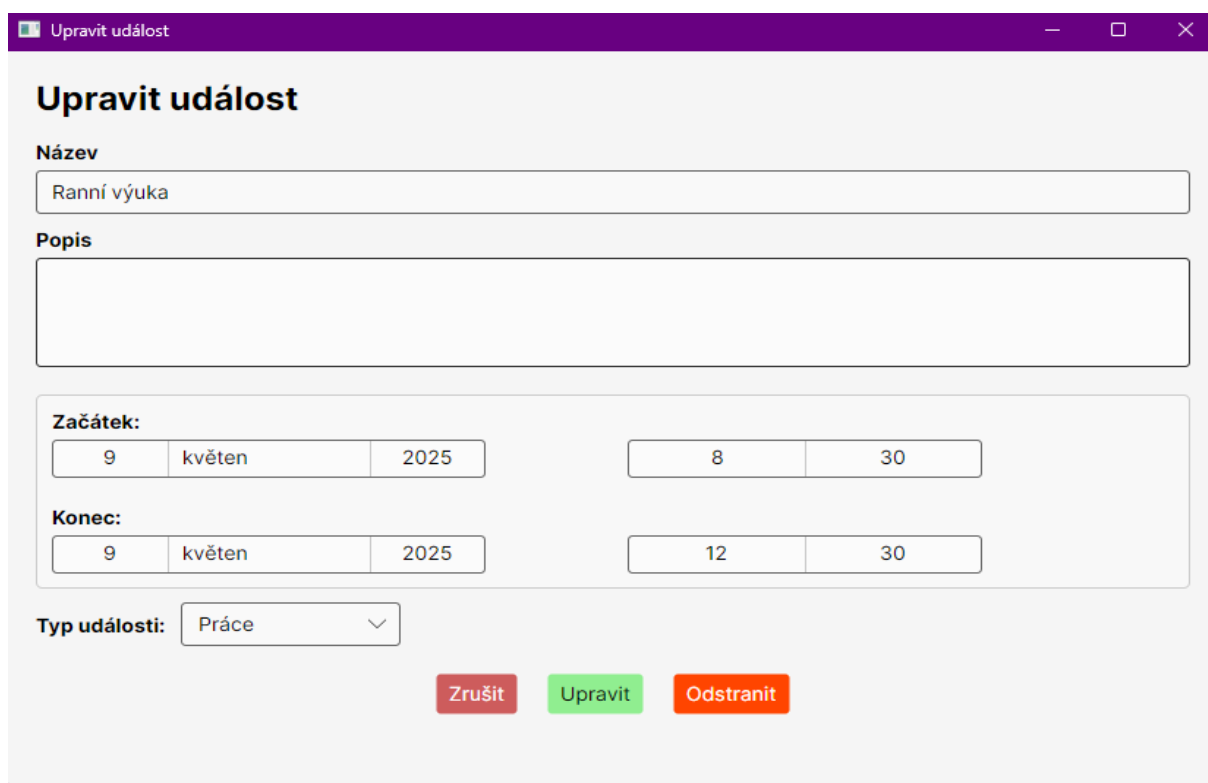
**Typ události:** Práce

Zrušit Vytvořit

Obrázek 11 Dialogové okno pro vytvoření

### 2.6.3 Dialogové okno pro změnu události

Dialogové okno pro změnu události obsahuje následující prvky. První pole slouží k zadání nebo změně názvu události. Toto pole je povinné, při pokusu o změnu události bez názvu se zobrazí dialogové okno s chybou. Druhé pole není povinné a slouží k přidání popisu události. Dále následuje blok pro nastavení data a času události. Za blokem pro nastavení data se nachází prvek, který umožňuje vybrat typ události (pracovní, volno, služební cesta atd.). Posledními prvky jsou tlačítka pro zrušení, které zavře okno, tlačítko pro uložení změn, které aktualizuje událost a zavře okno, a tlačítko, které slouží k odstranění události. Před odstraněním události bude uživatel požádán o potvrzení odstranění události.



**Upravit událost**

**Název**

Ranní výuka

**Popis**

**Začátek:**

9 květen 2025 8 30

**Konec:**

9 květen 2025 12 30

**Typ události:** Práce

Zrušit Upravit Odstranit

Obrázek 12 Dialogové okno pro změnu

#### 2.6.4 Vizuální prezentace pro den

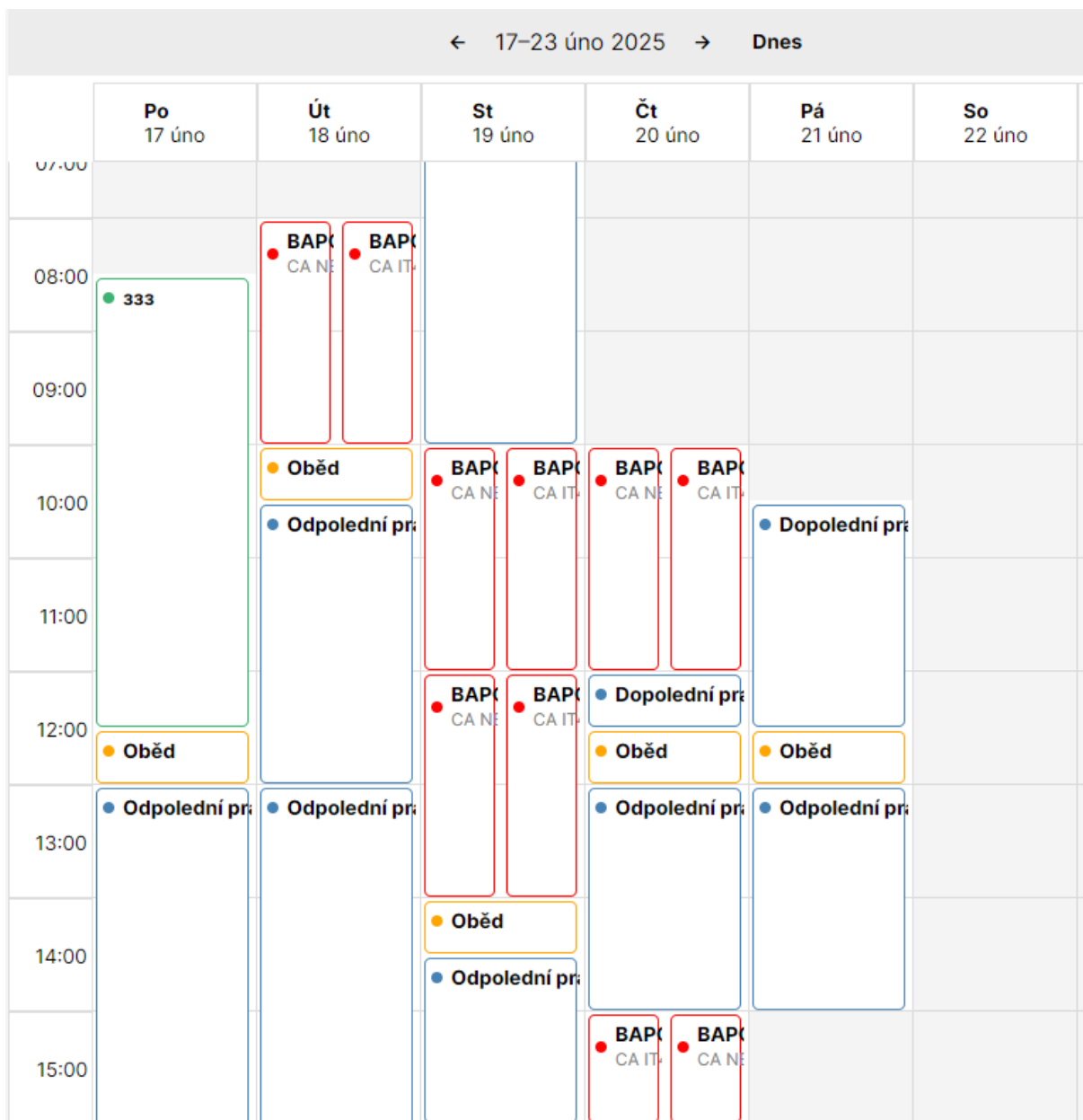
Vizuální prezentace pro daný den obsahuje mřížku s hodinami, pracovní interval je barevně odlišen od nepracovního. V horní části jsou tlačítka, která umožňují přepínat mezi dny. Tam se také nachází tlačítko, které přesune uživatele na aktuální den. Stejně tak zobrazuje události pro daný den, pokud nějaké existují.

| ← Den: 20 úno 2025 → Dnes |                                                                      |
|---------------------------|----------------------------------------------------------------------|
| 04:00                     |                                                                      |
| 05:00                     |                                                                      |
| 06:00                     |                                                                      |
| 07:00                     |                                                                      |
| 08:00                     |                                                                      |
| 09:00                     |                                                                      |
| 10:00                     | <div>● <b>BAPO Cvičení</b><br/>CA NET102</div>                       |
| 11:00                     | <div>● <b>BAPO Cvičení</b><br/>CA IT401</div>                        |
| 12:00                     | <div>● <b>Dopolední pracovní doba</b></div> <div>● <b>Oběd</b></div> |
| 13:00                     | <div>● <b>Odpolední pracovní doba</b></div>                          |
| 14:00                     |                                                                      |
| 15:00                     | <div>● <b>BAPOW Cvičení</b><br/>CA IT401</div>                       |
| 16:00                     | <div>● <b>BAPOW Cvičení</b><br/>CA NET101</div>                      |
| 17:00                     |                                                                      |
| 18:00                     |                                                                      |

Obrázek 13 Vizuální prezentace pro den

### 2.6.5 Vizuální prezentace pro týden

Vizuální prezentace pro týden obsahuje také tabulku s hodinami a dny, které tvoří týden. V horní části jsou tlačítka, která umožňují přepínat mezi týdny. Tam se také nachází tlačítko, které přesune uživatele na aktuální týden. Stejně tak zobrazuje události pro tento týden, pokud nějaké jsou.



Obrázek 14 Vizualizace prezentace pro týden

## 2.6.6 Vizualizace prezentace pro měsíc

Vizualizace prezentace pro měsíc obsahuje také mřížku, ale již s dny, které spadají do tohoto měsíce. Státní svátky, víkendy a dny z předchozího nebo následujícího měsíce se liší barvou pozadí nebo barvou čísla dne. V horní části jsou tlačítka, která umožňují přepínat mezi měsíci. Tam se také nachází tlačítko, které přesune uživatele na aktuální měsíc. Stejně tak zobrazuje události pro tento měsíc, pokud nějaké jsou.

| ← únor 2025 → Dnes               |                                  |                                    |                                    |                                  |    |    |
|----------------------------------|----------------------------------|------------------------------------|------------------------------------|----------------------------------|----|----|
| Po                               | Út                               | St                                 | Čt                                 | Pá                               | So | Ne |
| 27<br>08:00 Dopole<br>12:00 Oběd | 28<br>08:00 Dopole<br>12:00 Oběd | 29<br>08:00 Dopole<br>12:00 Oběd   | 30<br>08:00 Dopole<br>12:00 Oběd   | 31<br>08:00 Dopole<br>12:00 Oběd | 1  | 2  |
| 3<br>08:00 Dopole<br>12:00 Oběd  | 4<br>08:00 Dopole<br>12:00 Oběd  | 5<br>08:00 Dopole<br>12:00 Oběd    | 6<br>08:00 Dopole<br>12:00 Oběd    | 7<br>08:00 Dopole<br>12:00 Oběd  | 8  | 9  |
| 10<br>08:30 Dopole<br>12:30 Oběd | 11<br>08:00 BAPO<br>08:00 BAPO   | 12<br>09:30 Dopole<br>10:00 BAPO C | 13<br>08:30 Dopole<br>10:00 BAPO C | 14<br>08:30 Dopole<br>12:30 Oběd | 15 | 16 |
| 17<br>08:30 333<br>12:30 Oběd    | 18<br>08:00 BAPO<br>08:00 BAPO   | 19<br>06:00 baby<br>10:00 BAPO C   | 20<br>10:00 BAPO C<br>10:00 BAPO C | 21<br>10:30 Dopole<br>12:30 Oběd | 22 | 23 |
| 24<br>08:30 Dopole<br>12:30 Oběd | 25<br>08:00 BAPO<br>08:00 BAPO   | 26<br>09:30 Dopole<br>10:00 BAPO C | 27<br>08:30 Dopole<br>10:00 BAPO C | 28<br>08:30 Dopole<br>12:30 Oběd | 1  | 2  |
| 3<br>08:30 Dopole<br>12:30 Oběd  | 4<br>08:00 BAPO<br>08:00 BAPO    | 5<br>09:30 Dopole<br>10:00 BAPO C  | 6<br>08:30 Dopole<br>10:00 BAPO C  | 7<br>08:30 Dopole<br>12:30 Oběd  | 8  | 9  |

Obrázek 15 Vizuální prezentace pro měsíc

### 2.6.7 Globální nastavení

Globální nastavení obsahuje velké množství polí pro zadání informací o různých časových intervalech a dalších informací o uživateli, které budou použity pro vytváření automatických událostí, generování PDF a tak dále. Podrobný popis začneme tlačítky, která slouží k přepínání mezi zimním a letním semestrem, což uživateli umožňuje zadat různé údaje pro konkrétní semestr. Pole pro zadávání informací jsou stejná pro obě varianty. Dále následuje blok, který odpovídá za globální interval pracovní doby. Poté následuje blok s možností zadání informací o příchodu do práce a odchodu z práce a také času přestávky. Tato pole jsou stejná pro všechny pracovní dny. Záložka další nastavení obsahuje pole pro zadání jména, katedry, které budou použity pro generování v PDF. Pole pro zadání omezení pro přestávku a také jména, která se používají pro vytváření automatických událostí. Na konci se také nachází tlačítko pro uložení.

[← Zpět ke kalendáři](#)

Rok 2025 · Letní semestr

Rok: 2025

☐ Zimní

☒ Letní

Globální pracovní doba · Letní semestr

Od: 08:30

Do: 17:00

Pracovní doba · Letní semestr

Pondělí

Úterý

Středa

Čtvrtek

Pátek

Příchod: 08:30

Odchod: 17:00

Oběd, začátek: 12:30

Oběd, konec: 13:00

Další nastavení

Údaje pro PDF

Jméno: Radek Matoušek

Útvar: Katedra informačních technologií

Pauzy

Min. délka pauzy: 00:15

Max. délka pauzy: 01:00

Názvy událostí

Před obědem: Dopolodní pracovní doba

Pro oběd: Oběd

Po obědě: Odpolední pracovní doba

Uložit nastavení

Obrázek 16 Globální nastavení

### 2.6.8 Ukázka PDF

Náhled PDF obsahuje dva prvky, se kterými lze interagovat. První slouží k přepínání mezi měsíci, druhý k uložení souboru do počítače. Také obsahuje obrázek, jak bude stažený soubor vypadat.

44

## Evidence pracovní doby, včetně přestávek v práci a práce přesčas

Fakulta elektrotechniky a informatiky  
 jméno: Radek Matoušek  
 útvar: Katedra informačních technologií

pracovní doba: 08:30-17:00 hod.  
 docházka za měsíc: červenec 2025

Fond prac. doby:  
 184 hodin

| Den    | Začátek<br>pracovní doby | 1. přestávka |       | 2. přestávka |    | Konec<br>pracovní<br>doby | Odpracováno | Přesčas  | Neodprac. | Poznámka<br>*) |
|--------|--------------------------|--------------|-------|--------------|----|---------------------------|-------------|----------|-----------|----------------|
|        |                          | Od           | Do    | Od           | Do |                           |             |          |           |                |
| 1.     | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 2.     | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 3.     | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 4.     | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 5.     |                          |              |       |              |    |                           |             |          |           | S              |
| 6.     |                          |              |       |              |    |                           |             |          |           | S              |
| 7.     | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 8.     | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 9.     | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 10.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 11.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 12.    |                          |              |       |              |    |                           |             |          |           |                |
| 13.    |                          |              |       |              |    |                           |             |          |           |                |
| 14.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 15.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 16.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 17.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 18.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 19.    |                          |              |       |              |    |                           |             |          |           |                |
| 20.    |                          |              |       |              |    |                           |             |          |           |                |
| 21.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 22.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 23.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 24.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 25.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 26.    |                          |              |       |              |    |                           |             |          |           |                |
| 27.    |                          |              |       |              |    |                           |             |          |           |                |
| 28.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 29.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 30.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| 31.    | 08:30                    | 12:30        | 13:00 |              |    | 17:00                     | 08:00:00    | 00:00:00 | 00:00:00  |                |
| Celkem |                          |              |       |              |    |                           | 184:00:00   | 00:00:00 | 00:00:00  |                |

kontr. č. Σ odprac. - přesčas + neodprac. hodin (odpovídá FPD): 184:00:00

podpis zaměstnance:

podpis nadřízeného pracovníka:

\*) do poznámky: D - dovolená, N - nemoc, OČR - ošetřování, L - lékař, PC - pracovní cesta, S - svátek a ostatní dny pracovního klidu

Obrázek 17 Ukázka PDF

## 2.7 Algoritmy

Tato část popisuje, jak jsou v aplikaci řešené dva klíčové procesy: vyrovnávání hodin podle pravidel EPD a import rozvrhu ze souboru CSV. Popis vychází z aktuální implementace a odpovídá třídám a službám v projektu.

### 2.7.1 Algoritmus vyrovnávání hodin

Cílem je dorovnat 8 h na den / 40 h na týden a zároveň držet měsíční fond, aniž by se posouvaly ruční události. Úpravy se dělají jen přes automatické pracovní bloky na krajích dne. Celý proces řídí třída *EventService*.

Nejprve proběhne příprava měsíce v metodě *BalanceEventsForMonthAsync*. Z vybraného roku a měsíce se sestaví seznam pracovních dnů (bez víkendů a svátků přes *HolidayHelper*) a ten se rozdělí do ISO týdnů. Každý týden zpracuje *BalanceOneWeekAsync*. Pokud jsou první a poslední týden v měsíci neúplné, vyváží se ještě společně přes *BalanceTwoWeeksAsync*, aby seděl měsíční fond. Na závěr běží *PostNormalizeMonthAsync*, která doladí chybějící minuty a zkontroluje oběd.

Pro výpočty denních hodnot používá kód službu *WorkingHoursCalculatorService*. Ta v *DailyMetrics* oddělí práci (běžná práce + služební cesty) od zvláštních stavů (dovolená, nemoc,

OČR, lékař). Zvláštní stavy mají přednost, takže případné překryvy s výukou/prací se do odpracovaného času nezapočítají. *BalanceOneWeekAsync* si z těchto metrik sestaví pro každý den dvojici *Need/Extra* (kolik chybí do 8 h, kolik je nad).

Nejprve se přesčas řeší v tom samém dni: *TrimOvertimeOnSameDayAsync* volá *TrimAutoSymmetricAsync*, která symetricky zkracuje automatické bloky na začátku a konci dne (*TrimStartAuto*, *TrimEndAuto*). Tyto metody vždy upraví jen auto-bloky typu Work, zároveň aktualizují uživatelská nastavení dne (příchod/odchod, oběd) a následně srovnají oběd přes *ReplaceLunchEvent* a *SplitAutoWorkAroundLunch*, aby přestávka neležela přes okraj. Po zásahu se den regeneruje (*AutomaticEventsGeneratorService.RegenerateDailyEventsAsync*).

Pokud je den „uzamčený“ ručními okraji (první i poslední pracovní blok je ruční), rozpozná to *IsLockedEdgeDay*. Přesčas z takového dne se přenáší do jiných dnů týdne metodou *TransferLockedOvertimeAsync*. Ta nejdřív zjistí „kapacitu k řezu“ na začátku a na konci u ostatních dnů (*GetStartCap*, *GetEndCap*) a spravedlivě rozděljuje, kolik, kdo zkrátí (*TrimStartAuto*, *TrimEndAuto*). Všechny přesuny se zapisují do *WorkTransferReportingService*, aby byly dohledatelné.

Zbylé deficity a přebytky páruje *BalanceOneWeekAsync* pomocí jednoduchých front. Dny s přebytkem „darují“ čas dnům s deficitem, ale opět jen přes automatické bloky: dárce se zkracuje konec a pak začátek (*TrimEndAuto/TrimStartAuto*), příjemci se prodlužuje konec a pak začátek (*ExtendEndAuto/ExtendStartAuto*). Po každém kroku se oba dny regenerují a srovná se oběd (*EnsureLunchInsideWorkWindowAsync*). Pokud už není možné párovat jedna-ku-jedné, doladí se flexibilní dny s volným auto-okrajem krátkým prodloužením nebo zkrácením.

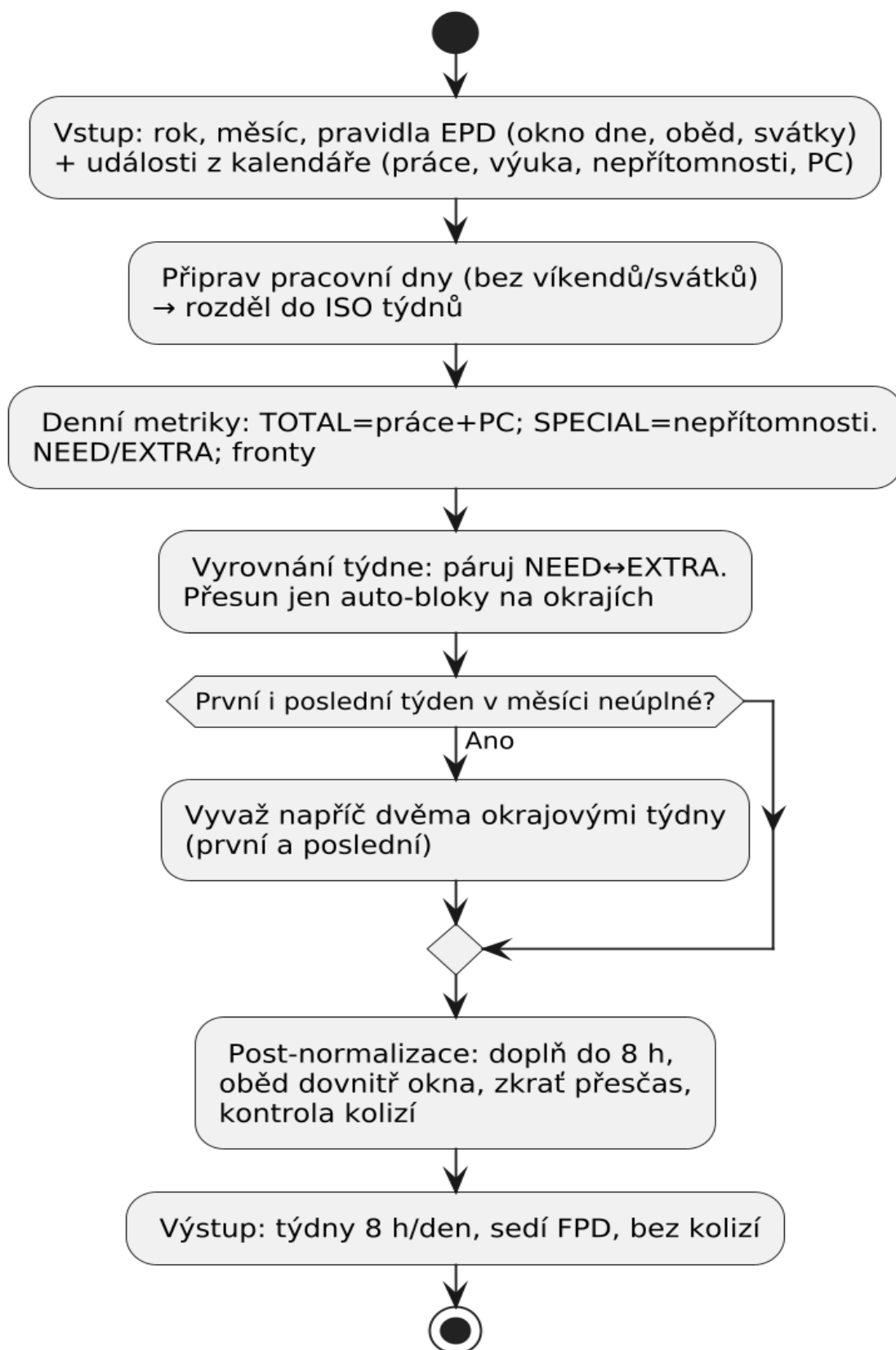
Zacházení s obědem obstarává *EnsureLunchInsideWorkWindowAsync*. Nejprve zkusí přestávku znovu umístit dovnitř okna příchod–odchod metodou *RefitLunch*, která si spočítá volná okna mezi obsazenými intervaly (*GetBusyIntervals*) a vybere vhodné místo co nejbližší středu dne. Když se oběd po úpravách nikam nevejde, kód se jej pokusí vyříznout z vnitřního auto-bloku (*TryCarveLunchFromInnerAuto*). Pokud ani to nejde, rozumně rozšíří okraje (*ExtendStartAuto/ExtendEndAuto*) a zkusí umístění znovu. Všechny změny končí zápisem do *UserSettings* a nahrazením odpovídající události oběda (*ReplaceLunchEvent*), aby se v přehledech zobrazoval skutečný stav.

Pro některé scénáře je potřeba „řezat“ spojené pracovní úseky na konci dne bez ohledu na vlastníky. K tomu slouží *TrimDonorFromEndByUnionAsync*, která si nejprve složí sjednocené segmenty práce v rámci okna dne (*BuildMergedWithOwners*) a pak posouvá konce dotčených

událostí tak, aby výsledný součet klesl o požadovaný čas. I zde po úpravách následuje srovnání oběda a re-generace dne.

Konzistenci po uživatelských zásazích hlídá *AdjustUserSettingsAfterChange*. Vypočte nové okraje podle skutečných pracovních bloků v daný den, prověří, zda oběd leží uvnitř okna, a podle potřeby jej nahradí. Každý zásah invaliduje měsíční cache (*InvalidateMonthsForDates*) a přes *MessageBus* rozšíří zprávy, aby se UI přepočítalo.

Díky těmto krokům je chování algoritmu předvídatelné: ruční události zůstávají na místě, upravují se jen auto-okraje, oběd je vždy uvnitř okna a kolize se nevytvářejí. Stejně vstupy dávají stejný výsledek a opakované spuštění nad vyrovnanými daty (*TrimOvertimeByAutoBlocksAsync*, *PostNormalizeMonthAsync*) už další změny nevyvolá.



Obrázek 18 Blokový diagram algoritmu

### 2.7.2 Algoritmus importu rozvrhu ze STAG

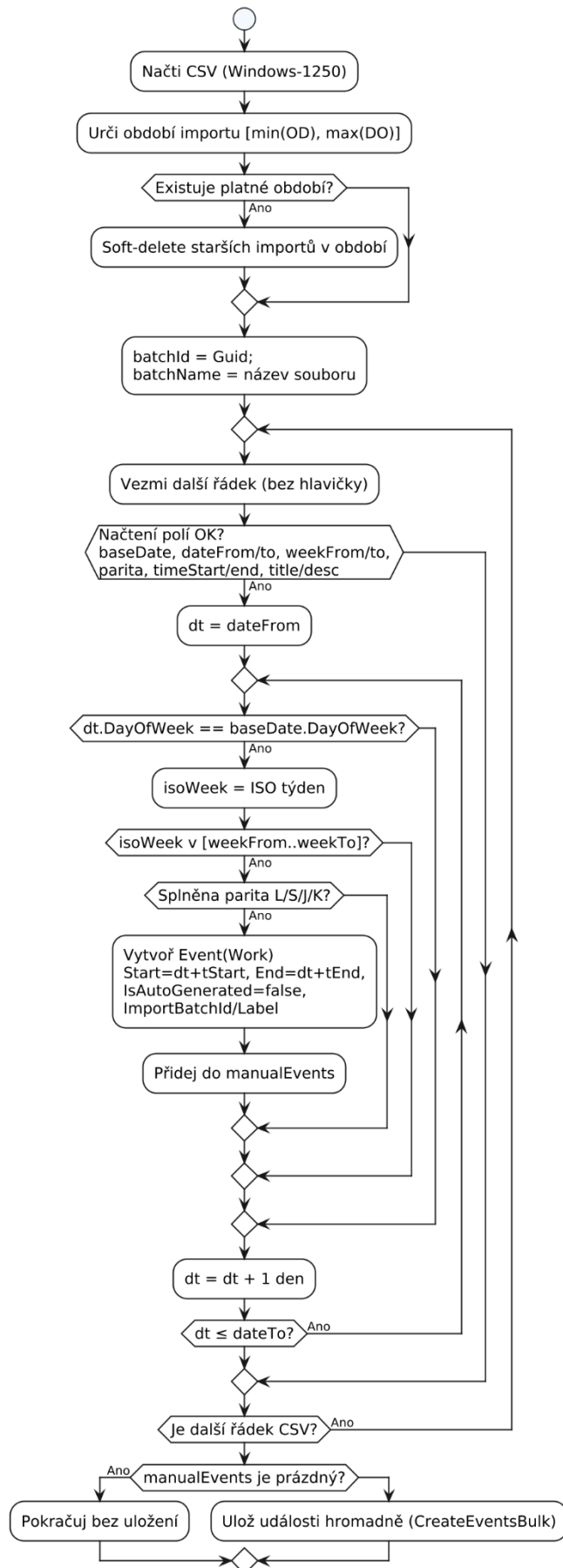
Cílem importu je převést řádky z CSV do manuálních pracovních událostí a hned po uložení z nich sestavit použitelný den pro EPD – tedy doplnit automatické okraje dne a oběd, srovnat kolize a případné přesčasy na hranách. Vstupem je cesta k souboru CSV (očekává se Windows-1250, hodnoty v uvozovkách a běžný oddělovač); výstupem je seznam vytvořených událostí typu Work bez příznaku auto-generace.

Import nejprve projde všechny řádky a určí nejmenší „od“ a největší „do“. V tomto zjištěném období pak provede soft-delete všech dříve importovaných záznamů, aby bylo možné bezpečně nahradit starou šarží novou (v kódu řeší *BulkSoftDeleteImportedInRangeAsync*). Následně řádek po řádku vytváří pracovní události: z CSV se čte základní den v týdnu, platnost od–do, časy začátku a konce, rozsah ISO týdnů a parita. Parita podporuje čtyři režimy – liché (L), sudé (S), každý týden (K) a „každý druhý s posunem“ (J), který bere ob týden, ale nezačíná prvním v intervalu. Pro každý den v platnosti se použije pouze ten, který odpovídá základnímu dni v týdnu, spadá do daného ISO rozsahu a projde danou paritou. Pokud projde, vznikne manuální Event s *EventType* = Work, nastavenými časy a původem (*ImportBatchId*, *ImportLabel*). Vše se sbírá do jedné kolekce a jednorázově uloží do databáze (*CreateEventsBulk*), aby se omezil počet transakcí.

Po uložení přichází „doladění“ dotčených dnů. Pro každý den, který import zasáhl, aplikace znovu vygeneruje automatické bloky a dohlédne na pauzu (*RegenerateDailyEventsAsync*). V této fázi se také řeší případné kolize přehledně přes callback s dotazem na uživatele. Potom se nastaví *UserSettings* daného dne: začátek odpovídá první skutečné práci, konec poslední práci (počítají se i služební cesty) a oběd se převezme z existujícího „Lunch“ záznamu; pokud chybí, použije se globální výchozí čas – záměrně jednoduché pravidlo, které lze později zpřesnit podle dne v týdnu. Nakonec se případná práce nad osm hodin zkrátí výhradně na automatických okrajích dne, ručně zadané bloky zůstávají nedotčené (*TrimOvertimeByAutoBlocksAsync*).

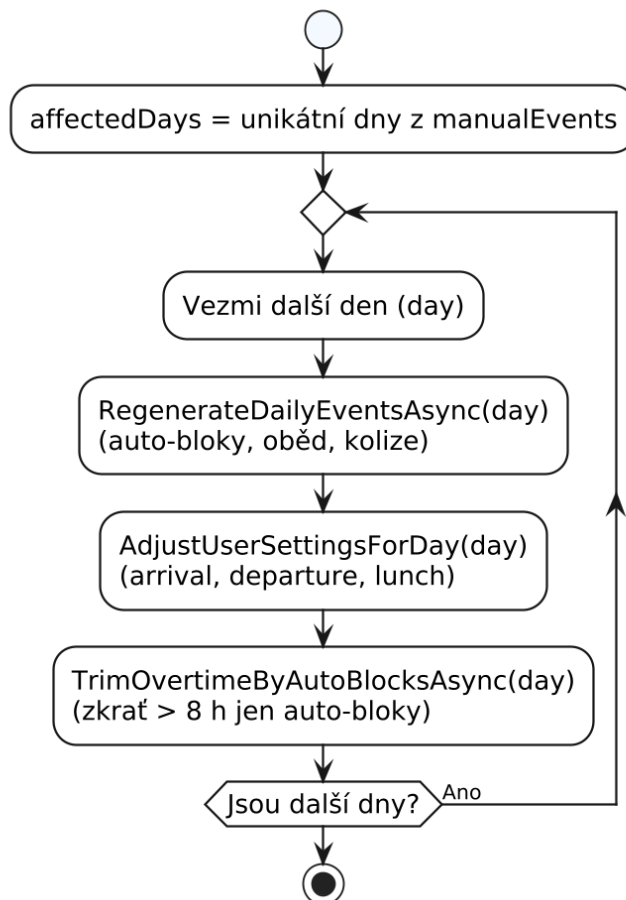
Důležité je, že import vytváří jen manuální „Work“ události – víkendy a svátky se do nich běžně nedostanou a vše okolo oběda, jemných dorovnání a hran dne přebírá automatický generátor. Po dokončení import odešle interní zprávy pro UI, aby se kalendář a náhledy hned aktualizovaly, a vrátí seznam nově založených pracovních událostí. Tento postup je rychlý, bezpečný při opakovaném spuštění a dává deterministický výsledek, který je připravený pro okamžitý export do PDF.

## Import rozvrhu ze CSV - část 1/3: Načtení a filtrování



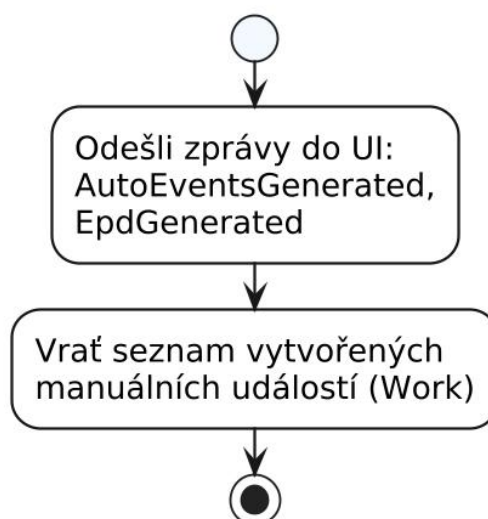
Obrázek 19 Blokový diagram algoritmu Importu 1/3

### Import rozvrhu ze CSV - část 2/3: Zpracování dotčených dnů



Obrázek 20 Blokový diagram algoritmu Importu 2/3

### Import rozvrhu ze CSV - část 3/3: Notifikace a návrat



Obrázek 21 Blokový diagram algoritmu Importu 3/3

## 2.8 Testování aplikace

Cílem testování bylo ověřit, že aplikace spolehlivě zvládá celý tok práce: od importu rozvrhu z CSV přes automatické vyrovnávání hodin až po export oficiální tabulky EPD do PDF. Testy probíhaly průběžně při vývoji i v závěru projektu jako ucelený běh se záznamem výsledků. Důraz byl na správné součty za dny, týdny a měsíc, čitelnost výstupů a stejné chování na Windows i Linuxu.

### 2.8.1 Cíl a kritéria přijetí

Aplikace je považována za přijatou, pokud:

- bez chyb načte CSV ze STAGu, založí události a přepočítá je podle pravidel EPD,
- vyrovná hodiny v rámci týdne a u okrajových týdnů měsíce,
- vytvoří PDF s korektní tabulkou EPD (hlavičky, svátky, součty),
- běží stejně na Windows i Linuxu a UI zůstává plynulé během dlouhodobějších operací.

### 2.8.2 Testovací prostředí

Aplikace byla testována na následujících operačních systémech a sadách nástrojů

- Windows 11 (23H2), .NET 8, Avalonia UI, SQLite (soubor)
- Ubuntu 22.04 LTS (GNOME, Wayland/X11), .NET 8, Avalonia UI, systémové fonty (DejaVu/Liberation)

### 2.8.3 Metodika

Použita kombinace integračních testů nad reálnou databází SQLite (Import → auto-události → vyrovnání → PDF), klikacích systémových testů v GUI a kontroly PDF (počet stran, hlavičky, zvýraznění svátků, součty týdnů i měsíce). Záměrně jsme zařadili i chybové vstupy (špatné kódování, chybějící sloupec, kolize). Binární shodu PDF jsme nepožadovali; porovnával se obsah.

### 2.8.4 Očekávané rozdíly mezi Windows a Linuxem

Linux používá jiné fonty a metriky, proto se může lišit šířka sloupců a zalomení. Vzhled prvků (rámečky, scrollbar) je také mírně odlišný. Požadavky: nic se nesmí překrývat, text musí být čitelný, ovládací prvky dostupné a **obsah PDF (texty a součty) musí být stejný**.

### 2.8.5 Scénáře (podrobný popis provedeního testu)

V testování používáme pojem **scénář** jako popis konkrétní situace: určíme vstup (např. soubor CSV nebo stav databáze), přesné kroky v aplikaci a očekávaný výsledek včetně kritéria, podle kterého poznáme, že test prošel. Každý scénář tedy běží podle stejného rámce — připravit data, vykonat akci v UI, ověřit chování v aplikaci a zkontrolovat, že součty i výstupy dávají smysl.

Nejprve ověřujeme běžný provoz. Vezmeme platný export CSV ze STAGu za jeden měsíc, spustíme import a po dokončení kontrolujeme počet vzniklých událostí, denní i týdenní součty a také to, že oběd byl vložen automaticky a případná práce nad 8 hodin je oříznuta pouze na auto-okraji dne. Na to navazuje scénář opakovaného importu téhož období: stará šarže se označí jako smazaná, duplicity nevznikají a v databázi zůstane jen poslední import.

Poté přicházejí negativní případy. Upravíme kódování či formát času v CSV tak, aby byl neplatný, spustíme import a čekáme řízenou chybu v UI; databáze se při tom nesmí změnit. Podobně vyzkoušíme CSV s chybějícím povinným sloupcem: import doběhne, problémové řádky se přeskočí s varováním v logu a uložená data jsou konzistentní. Dále simulujeme překryv dvou bloků výuky v jednom dni a ověřujeme, že se překryv nepočítá dvakrát, kolize je zřetelně označena a den nepřesáhne 8 hodin práce.

Následují situace, které prověřují pravidla EPD v širším kontextu. Pracujeme s neúplnými týdny — s měsícem se státním svátkem i se začátkem či koncem měsíce uprostřed týdne — a kontrolujeme, že týdenní požadavek odpovídá osmi hodinám na každý pracovní den a svátky odečítají osm hodin. Zvlášť testujeme vyrovnaní mezi prvním a posledním neúplným týdnem měsíce, přičemž plné týdny zůstávají beze změny. V samostatném scénáři kombinujeme služební cestu a výuku v jednom dni: služební cesta se započítá jako práce, překryv se nezdvouje a den se vejde do osmi hodin. Přidáváme i krátkou nepřítomnost přes výuku a sledujeme, že se správně projeví v položce „Neodpracováno“ a součty za týden sedí.

Do praktického provozu patří také ruční zásahy. Změníme okraj dne a délku oběda, očekáváme automatickou regeneraci auto-bloků, validní okraje a oběd uvnitř pracovního okna. Na konci řetězce ověřujeme export: vygenerujeme PDF za měsíc a kontrolujeme hlavičky, označení svátků, denní i týdenní součty a shodu obsahu mezi Windows a Linuxem.

Součástí testů je i výkon a použitelnost. Měříme čas importu i generování PDF na reálných datech a sledujeme, že operace proběhnou během několika sekund a UI zůstává responzivní (indikace průběhu, žádné „zamrznutí“). Zkoušíme škálování UI na 125–150 % a ověřujeme, že se prvky nepřekrývají a text je čitelný. Testujeme chybějící práva zápisu při ukládání DB nebo

PDF do chráněné složky — aplikace má zobrazit srozumitelnou hlášku a nabídnout změnu cesty. Ověřujeme také, že PDF obsahuje pouze dny daného měsíce, i když v datech existují záznamy mimo zvolené období. Dlouhou operaci (import) uprostřed běhu zrušíme a sledujeme bezpečné ukončení i konzistenci databáze. Nakonec spouštíme opakovaný import téhož CSV a porovnáváme výsledek: musí být shodný, aby byla potvrzena stabilita a determinismus.

## 2.8.6 Souhrnná tabulka testů

| ID | Platforma | Scénář                                                                       | Akce systému (zkráceně)                                                                                                                              | Výsledek testu                                              |
|----|-----------|------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| 1  | Win/Linux | Import platného CSV ze STAGu; kontrola denních/týdenních součtů a auto-oběda | Načte CSV → určí období → soft-delete staré šarže → vytvoří Work události → pro dotčené dny vygeneruje auto-bloky a oběd, zkrátí >8 h jen auto-bloky | PASS – dny vyplněny, součty sedí, bez chyb                  |
| 2  | Win/Linux | Opakovaný import téhož období; bez duplicit                                  | Najde stejné období a označí staré záznamy jako smazané → uloží novou šarži                                                                          | PASS – bez duplicit; v DB je jen aktuální šarže             |
| 3  | Win/Linux | CSV s chybně formátovaným datem/časem                                        | Nepodaří se číst datum/čas → vrátí řízenou chybu a zobrazí zprávu                                                                                    | PASS – aplikace nespadne; uživatel dostane vysvětlení       |
| 4  | Win/Linux | CSV bez povinného sloupce                                                    | Kontrola délky a povinných indexů → řádky přeskočí, zalogue varování                                                                                 | PASS – import doběhne; problémové řádky uvedeny             |
| 5  | Win/Linux | Překryv dvou bloků výuky v jeden den                                         | Při výpočtu se překryvy nepočítají dvakrát; kolize jsou označeny                                                                                     | PASS – za den nejvýše 8 h práce; kolize viditelné           |
| 6  | Win/Linux | Neúplné týdny (svátky, okraje měsíce)                                        | Požadavek týdne = 8 h × počet pracovních dnů (svátky/volno odečteny)                                                                                 | PASS – týden se svátkem = 32 h; zač./konec měsíce v pořádku |
| 7  | Win/Linux | Vyrovnaní mezi 1. a posledním neúplným týdnem                                | Spustí párové vyrovnaní mezi prvním a posledním neúplným týdnem                                                                                      | PASS – přenos hodin proběhl; plné týdny beze změny          |
| 8  | Win/Linux | Služební cesta + výuka v jeden den                                           | PC se započítá jako práce; překryv s výukou se nezdojuje                                                                                             | PASS – den max. 8 h; priority dodrženy                      |
| 9  | Win/Linux | Krátká nepřítomnost přes výuku                                               | Do výkazu se zapíše nepřítomnost; práce v překryvu se nepočítá                                                                                       | PASS – „Neodpracováno“ s důvodem; součty sedí               |
| 10 | Win/Linux | Ruční editace + regenerace auto-bloků                                        | Po editaci se znovu vygenerují auto-bloky a                                                                                                          | PASS – okraje dne a oběd validní; bez kolizí                |

|    |           |                                      |                                                |                                                                       |
|----|-----------|--------------------------------------|------------------------------------------------|-----------------------------------------------------------------------|
|    |           |                                      | oběd; uloží se<br>UserSettings                 |                                                                       |
| 11 | Win/Linux | Export PDF za měsíc                  | Vytvoří tabulku za měsíc<br>se součty a svátky | PASS – obsah PDF<br>sedí na obou OS                                   |
| 12 | Win/Linux | Rychlost importu                     | Import měsíce ze souboru<br>ok.csv             | PASS – UI neblokuje;<br>čas <5 s                                      |
| 13 | Win/Linux | Rychlost PDF                         | Export měsíce do PDF                           | PASS – generace <3<br>s; soubor lze otevřít                           |
| 14 | Win/Linux | Škálování UI 125–150 %               | Zvětšení 125–150 %                             | PASS – nic se<br>nepřekrývá; text<br>čitelný                          |
| 15 | Win/Linux | Uložení do chráněné<br>složky        | Pokus uložit DB/PDF do<br>chráněné složky      | PASS – zobrazeno<br>srozumitelné<br>hlášení; nabídnuta<br>změna cesty |
| 16 | Win/Linux | Data mimo zvolený<br>měsíc           | PDF obsahuje jen daný<br>měsíc                 | PASS – v PDF nejsou<br>navíc dny; součty<br>správně                   |
| 17 | Win/Linux | Zrušení importu                      | Bez pádu, DB konzistentní                      | PASS – bezpečné<br>ukončení; DB v<br>pořádku                          |
| 18 | Win/Linux | Determinismus<br>opakovaného importu | Stejně CSV → stejný<br>výsledek                | PASS – výsledek<br>stejný; bez rozdílů                                |

### 2.8.7 Závěr a známá omezení

Import, vyrovnání i export PDF se chovají shodně na Windows i Linuxu. Vizuální rozdíly na Linuxu jsou drobné a netýkají se dat. PDF se může lišit v metadatech (čas vytvoření), proto porovnáváme obsah, ne binární soubory. Pokud bude v rámci oponentury potřeba doplnit ještě detailnější postupy, lze k jednotlivým scénářům připojit testovací CSV a screenshoty kontrolních obrazovek.

## ZÁVĚR

Cílem práce bylo navrhnout, vytvořit a ověřit multiplatformní desktopovou aplikaci, která vyučujícím zjednoduší evidenci pracovní doby. Aplikace načte rozvrh ze STAGu, převede ho do jednotné podoby, dopočítá přestávky a v povolených mezích upraví pouze automatické okraje dne. Ruční záznamy nechává na místě. Při výpočtech respektuje svátky, nepřítomnosti a pravidla EPD pro týdenní i měsíční součty. Výstupem je přehledná tabulka připravená k exportu do PDF.

Použitý technologický základ je jednoduchý a dobře provázaný: C# a .NET pro logiku, Avalonia UI s MVVM pro uživatelské rozhraní, ReactiveUI pro reaktivní stav a příkazy a SQLite s Entity Framework Core pro lokální úložiště. Model je záměrně úsporný. Hlavní tabulka Events nese všechny denní události, GlobalSettings poskytují výchozí pravidla a UserSettings umožňují jemné přenastavení konkrétních dnů. Modulární uspořádání projektu dělí aplikaci na jasné části: import CSV, generování a vyrovnávání událostí, tvorbu PDF a GUI.

Těžištěm řešení je algoritmus vyrovnávání hodin. Nejprve spočítá skutečnou práci a očekávanou normu, potom lokálně páruje přebytky a nedostatky a upravuje jen automatické bloky na začátku a na konci dne. Oběd se drží uvnitř pracovního okna. Pokud to nejde, algoritmus zkusí přestávku vyříznout z vnitřního auto-bloku nebo rozumně upraví okraje. Plné týdny se vyrovnávají uvnitř sebe, neúplné týdny na okrajích měsíce lze dorovnat mezi sebou, aby seděl měsíční fond. Celý postup je stabilní a průhledný: stejné vstupy dávají stejný výsledek a každý zásah je dohledatelný.

V praktické části jsem popsal strukturu databáze, organizaci kódu a klíčové obrazovky. Aplikace nabízí denní, týdenní i měsíční pohled, rychlý náhled PDF a jednoduchý export. Díky Avalonii běží na Windows i Linuxu a chová se stejně. Testování pokrylo běžné i hraniční situace: import a opakovaný import CSV, chybné vstupy, kolize, svátky a neúplné týdny, párové dorovnání okrajů měsíce, kombinaci výuky a služební cesty, ruční úpravy s následnou regenerací, výkon importu a tisku, škálování UI, práci bez práv zápisu i bezpečné zrušení dlouhé operace. Výsledky potvrzují správné součty, čitelné výstupy a stabilní chování na obou platformách.

Hotová aplikace zkracuje čas na měsíční přehledy, snižuje chybovost a zpřehledňuje správu pracovního fondu. Do budoucna dává smysl přidat jednotné přihlášení, auditní logování a další exportní formáty. Tyto kroky posílí použitelnost, správu přístupů a otevřou cestu k dalšímu rozšíření.

## POUŽITÁ LITERATURA

1. **MICROSOFT**, 2025. *A tour of the C# language*. In: **learn.microsoft.com** [online]. Microsoft, 2025 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/cs-cz/dotnet/csharp/tour-of-csharp/overview>
2. **MICROSOFT**, 2025. *.NET – Build modern apps and services* (úvod, multiplatformnost). In: **dotnet.microsoft.com** [online]. Microsoft, 2025 [cit. 2025-08-24]. Dostupné z: <https://dotnet.microsoft.com/en-us/>
3. **AVALONIA UI**, 2025. *The MVVM Pattern*. In: **docs.avaloniaui.net** [online]. Avalonia, 2025 [cit. 2025-08-24]. Dostupné z: <https://docs.avaloniaui.net/docs/concepts/the-mvvm-pattern/>
4. **REACTIVEUI COMMUNITY**, 2025. *Getting Started*. In: **reactiveui.net** [online]. ReactiveUI, 2025 [cit. 2025-08-24]. Dostupné z: <https://www.reactiveui.net/docs/getting-started/>
5. **SQLITE**, 2024. *SQL As Understood By SQLite*. In: **sqlite.org** [online]. SQLite, 2024 [cit. 2025-08-24]. Dostupné z: <https://www.sqlite.org/lang.html>
6. **QUESTPDF**, 2025. *QuestPDF – C# PDF Generation Library*. In: **questpdf.com** [online]. QuestPDF, 2025 [cit. 2025-08-24]. Dostupné z: <https://www.questpdf.com/>
7. **ELECTRON**, 2025. *What is Electron?* In: **electronjs.org/docs/latest** [online]. Electron, 2025 [cit. 2025-08-24]. Dostupné z: <https://electronjs.org/docs/latest/>
8. **PYTHON SOFTWARE FOUNDATION**, 2025. *Python 3.13 documentation*. In: **docs.python.org** [online]. PSF, 2025 [cit. 2025-08-24]. Dostupné z: <https://docs.python.org/>
9. **THE QT COMPANY**, 2025. *Qt for Python (PySide6) – Official documentation*. In: **doc.qt.io** [online]. Qt, 2025 [cit. 2025-08-24]. Dostupné z: <https://doc.qt.io/qtforpython-6/>
10. **JETBRAINS**, 2025. *Compose Multiplatform – Get Started*. In: **jetbrains.com** [online]. JetBrains, 2025 [cit. 2025-08-24]. Dostupné z: <https://www.jetbrains.com/help/kotlin-multiplatform-dev/compose-multiplatform.html>
11. **MICROSOFT**, 2023. *ASP.NET MVC Overview – Model–View–Controller*. In: **learn.microsoft.com** [online]. Microsoft, 2023 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/en-us/aspnet/mvc/overview/older-versions-1/overview/asp-net-mvc-overview>

12. **REACTIVEUI COMMUNITY**, 2025. *The Zen of the ViewModel*. In: **reactiveui.net** [online]. ReactiveUI, 2025 [cit. 2025-08-24]. Dostupné z: <https://www.reactiveui.net/docs/handbook/view-models/>
13. **CMU/SEI**, 2010. *What Is Your Definition of Software Architecture?* In: **sei.cmu.edu** [online]. Carnegie Mellon University, 2010 [cit. 2025-08-24]. Dostupné z: <https://www.sei.cmu.edu/library/what-is-your-definition-of-software-architecture/>
14. **MICROSOFT**, 2023. *Data binding a chování v Avalonia a WPF (Behaviors)*. In: **learn.microsoft.com** [online]. Microsoft, 2023 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/cs-cz/dotnet/desktop/wpf/advanced/behaviors-overview>
15. **AVALONIA XAML BEHAVIORS**, 2024. *GitHub – Avalonia.Xaml.Behaviors*. In: **github.com/AvaloniaXAMLBehaviors** [online]. Avalonia Team, 2024 [cit. 2025-08-24]. Dostupné z: <https://github.com/AvaloniaXAMLBehaviors>
16. **AVALONIA COMMUNITY**, 2024. *Behaviors v Avalonia UI*. In: **docs.avaloniaui.net** [online]. Avalonia Community, 2024 [cit. 2025-08-24]. Dostupné z: <https://docs.avaloniaui.net/docs/controls/behaviors>
17. **MICROSOFT**, 2023. *Common web application architectures – monolitická aplikace*. In: **learn.microsoft.com** [online]. Microsoft, 2023 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures>
18. **MICROSOFT**, 2025. *Microservices architecture style – přehled a trade-offy*. In: **learn.microsoft.com** [online]. Microsoft, 2025 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices>
19. **FOWLER, Martin**, 2015. *Monolith First*. In: **martinfowler.com** [online]. 2015 [cit. 2025-08-24]. Dostupné z: <https://martinfowler.com/bliki/MonolithFirst.html>
20. **AZURE ARCHITECTURE CENTER**, 2025. *Architecture styles – přehled*. In: **learn.microsoft.com** [online]. Microsoft, 2025 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/cs-cz/azure/architecture/guide/architecture-styles/>
21. **MICROSOFT**, 2024. *Entity Framework Core – SQLite provider*. In: **learn.microsoft.com** [online]. Microsoft, 2024 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/cs-cz/ef/core/providers/sqlite/>
22. **ARTIFEX SOFTWARE**, 2025. *Ghostscript – official documentation*. In: **ghostscript.com** [online]. Artifex Software, 2025 [cit. 2025-08-24]. Dostupné z: <https://www.ghostscript.com/>

23. **HABJAN, John** a kol., 2025. *Ghostscript.NET – .NET wrapper pro Ghostscript*. In: **github.com/jhabjan/Ghostscript.NET** [online]. 2025 [cit. 2025-08-24]. Dostupné z: <https://github.com/jhabjan/Ghostscript.NET>
24. **MICROSOFT**, 2025. *Visual Studio – Product documentation (Overview, IDE features)*. In: **learn.microsoft.com** [online]. Microsoft, 2025 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/cs-cz/visualstudio/ide/?view=vs-2022>
25. **SQLTESTUDIO**, 2025. *SQLiteStudio – Official website (Features, Downloads, Docs)*. In: **sqlitestudio.pl** [online]. 2025 [cit. 2025-08-24]. Dostupné z: <https://sqlitestudio.pl/>

## SEZNAM PŘÍLOH

Příloha A: Zdrojový kód aplikace TeacherScheduleApp

## **PŘÍLOHA A: Zdrojový kód aplikace TeacherScheduleApp**

Tato příloha obsahuje zip archiv s projektem z Visual Studio 2022 jménem TeacherScheduleApp